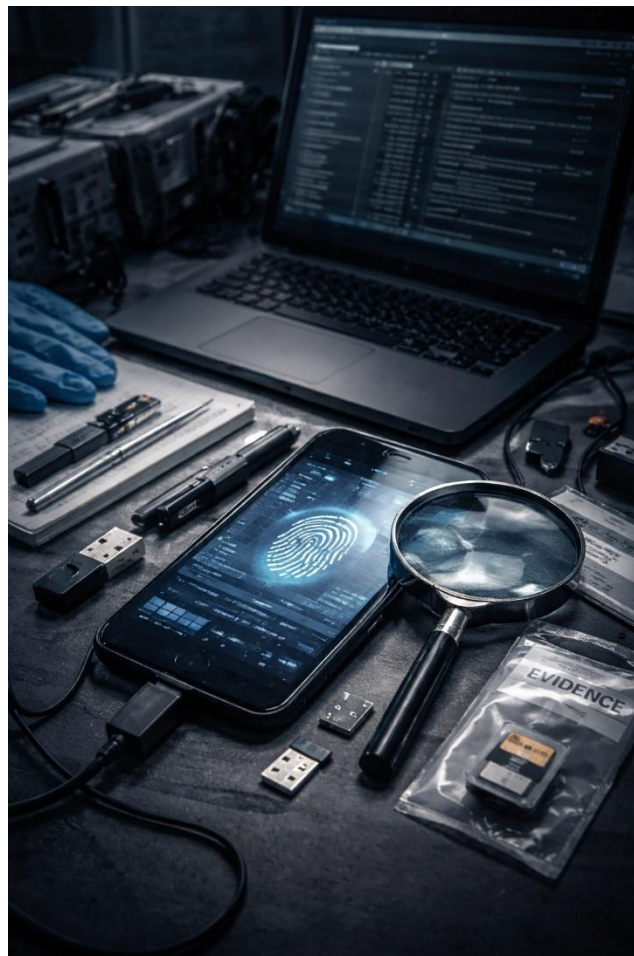


Análisis Forense Informático

Unidad 12. Análisis Forense de Dispositivos Móviles



Índice

1	Introducción.....	3
2	Investigación forense del dispositivo móvil.....	4
2.1	Entrada del dispositivo	4
2.2	Identificación	5
2.3	Preparación.....	5
2.4	Aislamiento	6
2.5	Procesamiento	7
2.6	Verificación	8
2.7	Informe y documentación	9
2.8	Archivado	9
3	Aproximaciones prácticas al forense de móviles.....	9
3.1	Comprender los diferentes sistemas operativos	9
3.2	Niveles de aplicación de la herramienta de adquisición	10
3.2.1	Extracción Manual.....	11
3.2.2	Extracción Lógica	11
3.2.3	Extracción de volcado binario (Hex Dump)	12
3.2.4	Lectura de memoria física sin extracción	12
3.2.5	Extracción del chip (Chip-off)	12
3.2.6	Micro Lectura	13
4	Evidencias potenciales en los dispositivos móviles.....	13
5	Particiones del sistema Android	14
6	ADB	18
6.1	Instalar y utilizar ADB	18
6.2	Comandos ADB	22
6.3	Duplicar pantalla del dispositivo en el equipo forense	25
7	Artefactos Android	26
7.1	SMS/MMS, Call log	26
7.2	Contactos y Addressbook	26
7.3	Historial del navegador	27
7.4	Bluetooth.....	28
7.5	Wifi	28
7.6	GPS	28
7.7	Fotos y video	29
7.8	Mensajería Whatsapp	29
7.9	Datos de aplicaciones	29
7.10	Uso de aplicaciones	30

7.11	Cuentas de usuarios y servicios de Google	30
7.12	Aplicaciones instaladas	30
7.13	Historial de descargas	31
7.14	Log de Eventos.....	31
8	Desbloqueo y acceso a un móvil Android.....	31
8.1	Antes de intentar desbloquear	32
8.2	Usar Android Device Manager para desbloquear el móvil (Android<v-5)	32
8.3	Saltar el bloqueo de pantalla	32
8.4	Usar ADB para intentar saltar la protección.....	33
8.5	Smudge Attack	34
8.6	Contraseña o patrón olvidado	34
8.7	Saltar bloqueo de aplicación de terceros	34
8.8	Usar una APK con permiso DISABLE_KEYGUARD	35
9	Códigos de control de móviles Android.....	35
10	IMEI.....	35
10.1	Obtener el IMEI.....	36
10.2	Utilidad del IMEI.....	37
10.3	Buscar información en Imei.info.....	37
11	Obtener información a partir de la tarjeta SIM	38
12	Extracción de datos lógica	38
12.1	Extracción con ADB pull.....	38
12.2	Extracción con ADB backup	40
12.3	Extracción con Android Studio	41
12.4	Extracción ADB dumpsys (extracción de datos en vivo)	41
12.5	Extracción ADB dumpsys (extracción de datos en vivo)	42
12.6	Triaje de Android.....	43
12.7	Usar Cellebrite, AXIOM, Mobiledit.....	44
13	Preguntas para guiar la investigación	44
13.1	Preguntas genéricas.....	44
13.2	Ejemplos de posibles escenarios.....	45

1 Introducción.

En el forense de móviles o smartphones existen una serie de problemas y retos a solucionar que hacen que sea mucho más complejo el análisis forense de estos dispositivos:

1. **Diferencias de Hardware:** Hay una gran variedad de dispositivos con hardware diferente.
2. **Sistemas operativos de móviles:** Los sistemas son variados: Android, Apple iOS y con muchas versiones.
3. **Características de seguridad de plataformas móviles:** Se aplica cifrado por defecto y protección de datos de los usuarios con ACL's.
4. **Adquisición difícil sin modificar datos:** En los móviles no basta con apagarlo porque muchos procesos se ejecutan en segundo plano. Por ejemplo, la aplicación de alarmas del sistema. En ocasiones apagar puede modificar el estado del móvil.
5. **Técnicas antiforense:** En los móviles se aplican técnicas "antiforense". Por ejemplo, se oculta la información, se ofusca, se borra los datos de forma segura, etc.
6. **Recuperar passwords:** Si el dispositivo está protegido por contraseña, es necesario acceder a él para recuperarla y saltar el bloqueo de pantalla. Existen métodos que no funcionan en todas las versiones.
7. **Falta de recursos:** Se necesitan muchas herramientas para que haya muchas casuísticas diferentes. Por ejemplo, accesorios como cables USB, baterías, cargadores de distintos móviles, etc.
8. **Naturaleza dinámica de las evidencias digitales:** Es muy fácil que se modifiquen por ejemplo navegando con un navegador. Es necesario anotar todo lo que se hace investigando para justificar los posibles cambios.
9. **Alteración del dispositivo:** Es posible que se haya modificado los datos de una aplicación moviéndolos de sitio o cambiando el nombre de los archivos del sistema. Hay que tener en cuenta el nivel de experiencia del sospechoso.
10. **Blindaje de las comunicaciones:** Los móviles utilizan redes celulares, Wifi, Bluetooth e infrarrojas para la comunicación (vía radio y señal infrarroja) y por tanto hay que aislar el dispositivo en bolsas de Faraday o tapando sensores.
11. **Falta de disponibilidad de herramientas:** Es necesaria una combinación de muchas herramientas ya que sólo una puede no soportar todos los modelos o tener todas las funciones necesarias. Elegir las herramientas adecuadas se complica.
12. **Programas maliciosos:** El dispositivo puede tener algún malware, como virus o troyanos. Este malware puede propagarse a otros dispositivos vía wifi, cable, etc.
13. **Problemas legales:** Al ser un dispositivo móvil, puede que el crimen se haya producido en diferentes zonas geográficas con legislación diferente.



2 Investigación forense del dispositivo móvil

El proceso de investigación tendría los siguientes pasos:

1. Entrada del dispositivo
2. Identificación
3. Preparación
4. Aislamiento
5. Procesado
6. Verificación
7. Documentación
8. Informe
9. Archivado

2.1 Entrada del dispositivo

A diferencia del PC, se puede requerir una actuación más rápida ya que:

- Se debe evitar la modificación o borrado remoto de evidencias. Por tanto hay que aislarlo en una caja o bolsa de Faraday.
- Es más probable que esté mojado por agua que un PC.

Si el móvil nos llega **mojado de agua**:

- Apagar y quitar baterías si es posible.
- Sumergirlo en alcohol isopropílico (isopropanol) o n-propanol durante unos minutos para sacar el agua
- Aplicar papel secante, o meterlo en un bote de avena o arroz.
- No se pueden utilizar secadores ni otras fuentes de calor.
- El agua salada es mucho más corrosiva que la dulce.

Cuando se recibe el dispositivo es necesario:

- Documentar el propietario.
- Determinar los objetivos de la investigación.
- Generar los documentos adecuados (entrada de pruebas, cadena de custodia).
- Respetar la ley.
- Podemos intentar desactivar el password o el bloqueo si el móvil está desbloqueado en ese momento.

2.2 Identificación

Hay que determinar en esta fase:

1. **Autoridades legales:** Qué autoridad legal se tiene para hacer el forense.
2. **Determinar los datos a extraer:** Así escoger las herramientas adecuadas.
3. **Información de identificación del dispositivo (modelo, versión, etc):** Recoger datos del dispositivo y estado físico (rayaduras, componentes adicionales, color, etc). Realizar fotografías del dispositivo.
4. **Dispositivos de almacenamiento adicionales:** tarjetas de memoria adicional que se pueden extraer para realizar una adquisición forense normal (imagen).
5. **Otras fuentes de evidencias:** huellas dactilares, etc. Es necesario llevar guantes al manipular los dispositivos.

2.3 Preparación

Para la preparación del dispositivo podemos encontrarnos con diferentes escenarios:

Teléfono encendido

Si el teléfono está desbloqueado:

- Es necesario garantizar alimentación eléctrica y aislar de la red.
- Extraeremos la tarjeta SIM (si se puede).
- Garantizaremos el acceso físico:
 - Desactivamos el código de bloqueo si es posible.
 - Activamos el modo debug USB.
 - Desactivamos o retrasamos el bloqueo de pantalla por inactividad.

Si el teléfono está bloqueado:

- La aislamos de la red
- Nos aseguramos de que el debug USB está activado
 - Si está activado, cargar un bootloader para cambiar el inicio y activar el acceso físico al terminal.
 - Si no, tratar de obtener el código de volqueo (smudge attack, fuerza bruta, vulnerabilidades, etc.)

A su vez, también trataremos de obtener tarjetas SD, backups en PC asociados, etc.

Teléfono apagado

En este caso es necesario:

- Extraer los dispositivos extraíbles.
- Encender el teléfono
- Actuar como con el dispositivo bloqueado.

En ambos casos los datos a obtener son los siguientes:

Es necesario buscar información sobre:

- El modelo de teléfono.
- Los métodos a utilizar por este modelo, por ejemplo para hacer root si es necesario.
- Buscar las herramientas de adquisición adecuadas.

La extracción y análisis dependen mucho de:

- Tipo de dispositivo (Android, Iphone).
- Versión de hardware y Software.
- Configuración del dispositivo.
- Nivel de bloqueo:
 - apagado/encendido
 - bloqueado o no
 - con root hecho o no
- Que datos son posibles adquirir y su volatilidad

2.4 Aislamiento

Dado que los dispositivos utilizan redes vía radio:

- Conectados a redes, **se puede modificar la información que contienen** (llamadas, mensajes, datos de aplicaciones, etc).
- También **es posible destruir los datos remotamente**.

Entonces hay que aislar el dispositivo básicamente de dos formas:

- **Activar Modo avión:** Hay que vigilar por que todavía puede recibir señales ya que en los aviones se permite wifi. No es posible si el dispositivo está bloqueado.
- Meterlo en una **Bolsa o caja de Faraday:** También hay tiendas y habitaciones de Faraday para trabajar a gusto. Bloquean la radiación electromagnética.



Bolsa del Faraday



"Tienda de Farady"

<https://www.teeltech.com/mobile-device-forensics-equipment/offgrid-faraday-bags/>

2.5 Procesamiento

Es necesario identificar las herramientas a usar: seleccionar por precio, facilidad de uso, aplicabilidad, etc.

El software forense es muy caro y normalmente se requieren varias herramientas para acceder a los datos:

- Es necesario asegurarse de que la herramienta permite asegurar la integridad.
- Es necesario usar un método probado y reproducible, así como "forensicamente" seguro.

Podemos realizar dos tipos de adquisiciones:

1. Adquisición física (imagen forense)

Es el método preferido. Se extrae una **imagen binaria de la memoria**.

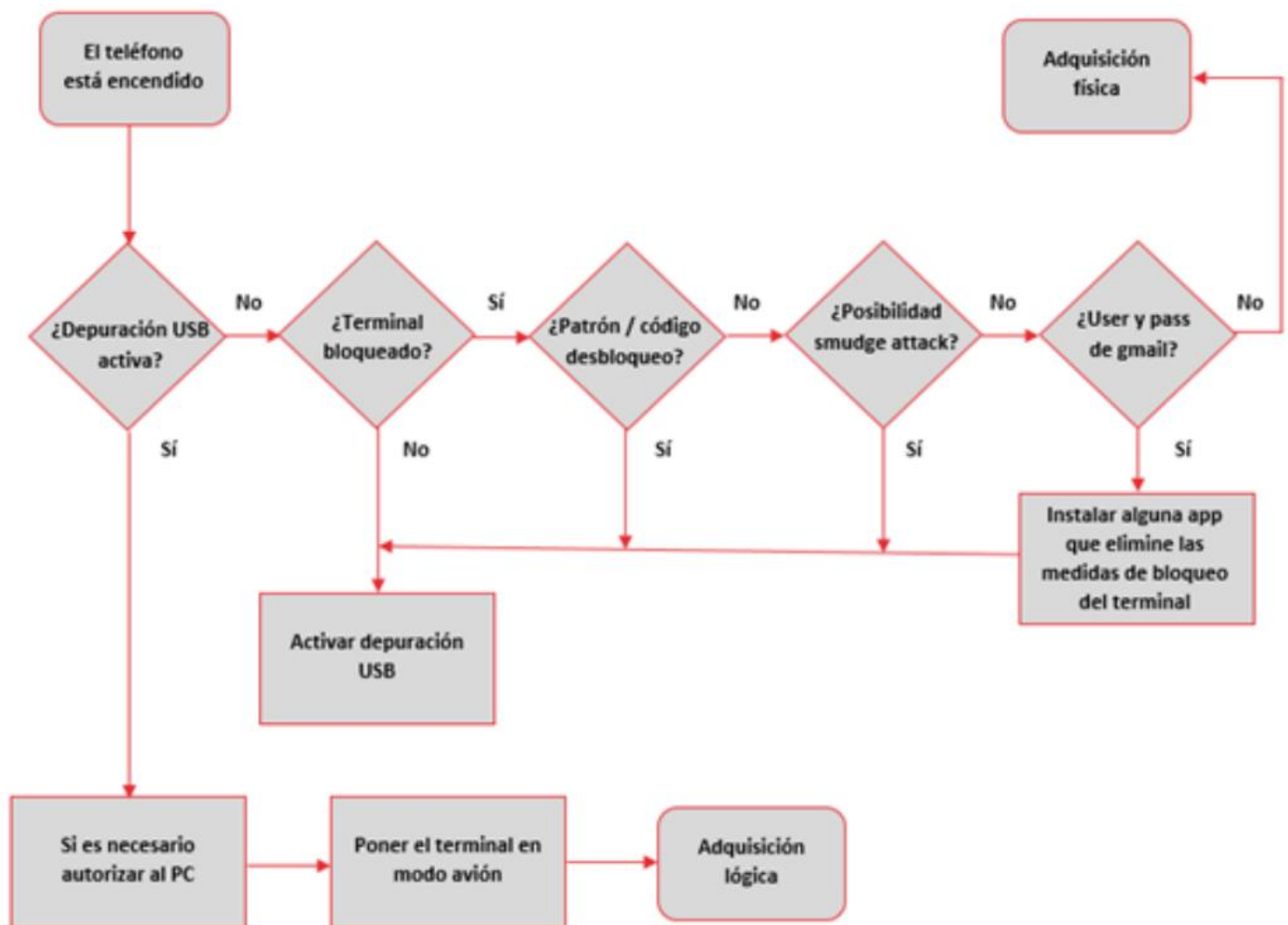
- Normalmente el móvil está apagado.
- Este método suele provocar pocos cambios en el dispositivo.

2. Adquisición lógica

Si no es posible o falla el método anterior, será la siguiente alternativa. De formes **siempre es aconsejable realizarla igualmente, aunque tengamos la adquisición física.**

- Se copia el sistema de archivos del dispositivo.
- Se debería realizar siempre ya que contiene información que permite guiar el análisis de la imagen física binaria.

Por último, debemos tener en cuenta las “trabas” que nos podemos encontrar (dispositivo bloqueado, sin depuración USB activada, ...). El siguiente diagrama indica la metodología de adquisición que debemos emplear y los problemas que nos podemos encontrar:



2.6 Verificación

Es necesario comprobar la exactitud de los datos adquiridos:

- **Comparando los datos extraídos con los del dispositivo.** Atención porque esto puede provocar cambios en su dispositivo.
- **Usar múltiples herramientas y comparar los resultados.** Sería ideal.
- **Usar funciones de hash,** ya que todas las imágenes deben tener uno. En el caso de imágenes lógicas, se calcula un hash para cada archivo.
- Para cualquier discrepancia entre el hash original y el de la copia, es necesario tener una justificación. Por ejemplo, si se ha vuelto a iniciar el dispositivo y se ha vuelto a adquirir una imagen, el hash puede cambiar.

2.7 Informe y documentación

Es necesario **documentar todo el proceso**, mientras se lleva a cabo. Hay que incluir:

- Fecha inicio y final.
- Estado físico del dispositivo (rayadas, color, etc.)
- Fotos del teléfono y componentes.
- Estado del teléfono (on-off)
- Modelo y versión del teléfono y sistema
- Herramientas usadas para la adquisición
- Herramientas usadas para examinar
- Datos encontrados al examinar
- Notas de los otros investigadores

Se debe hacer una **presentación clara de la información** por si debe leerlo otro investigador o por un juicio.

- Los hallazgos deben ser autoexplicativos y claros, reproducibles (que alguien otro pueda reproducir el proceso siguiendo la documentación).
- Las líneas de tiempo y los analizadores de enlaces pueden ayudar a explicar los hallazgos relacionando diferentes fuentes y dispositivos interconectados.

2.8 Archivado

En muchos casos hay que **guardar las evidencias** por intervalos de tiempo más o menos largos.

- Hay que tener presente los sistemas de archivado preservando las pruebas.
- En un futuro pueden aparecer nuevas tecnologías para analizar las imágenes binarias y visitar el caso.

3 Aproximaciones prácticas al forense de móviles

3.1 Comprender los diferentes sistemas operativos

Existen diferentes sistemas operativos para móviles como por ejemplo:

- iOS (2024 - The Apple Wiki - https://theapplewiki.com/wiki/Main_Page)
- Windows Phone (desaparecido hoy en día)
- Android (el que estudiaremos en profundidad)
- Otros

Android

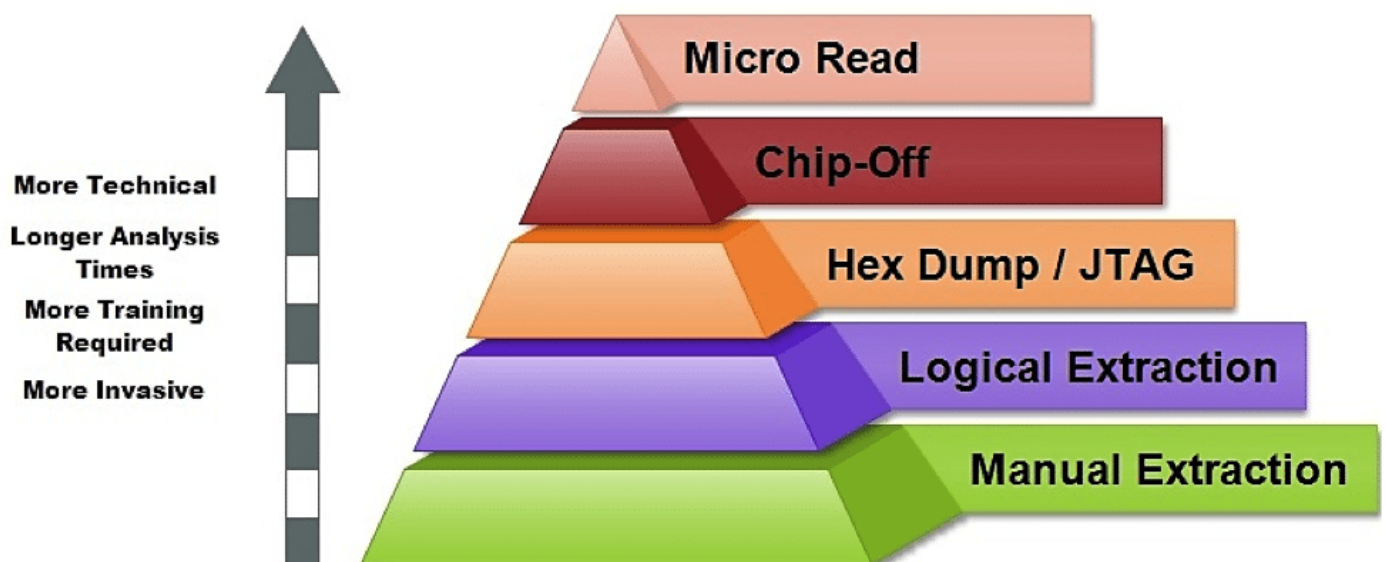
Sistema operativo basado en **Linux** y que es la plataforma open-source de Google. A los fabricantes de móviles, al ser open-source, el S.O. les sale gratis.

- Es el más usado en móviles en el mundo (iOS es el segundo).
- Su naturaleza abierta hace que existan muchísimas aplicaciones que normalmente se suben a Google Play (market de apps de Google).

Android	iOS	Samsung
70.86%	28.8%	0.33%
Unknown	Linux	Windows
0%	0%	0%

3.2 Niveles de aplicación de la herramienta de adquisición

La adquisición de los datos se puede hacer aplicando técnicas a diferentes niveles. Cada técnica tiene sus propias ventajas e inconvenientes.



Pirámide de nivelación de herramientas para teléfonos móviles (Sam Brothers, 2009)

Podemos categorizar las herramientas según el método de extracción que se aplique. Si subimos **desde la base de la pirámide hacia arriba aumenta:**

- el **nivel técnico** requerido del personal
- la **sofisticación** de las herramientas
- el **tiempo necesario** para llevarlas a cabo
- el **riesgo de destrucción** de datos y dispositivo.

Si se escoge el método erróneo, podemos destruir todos los datos o el propio dispositivo. Se requiere un buen entrenamiento en la técnica utilizada para evitar errores.

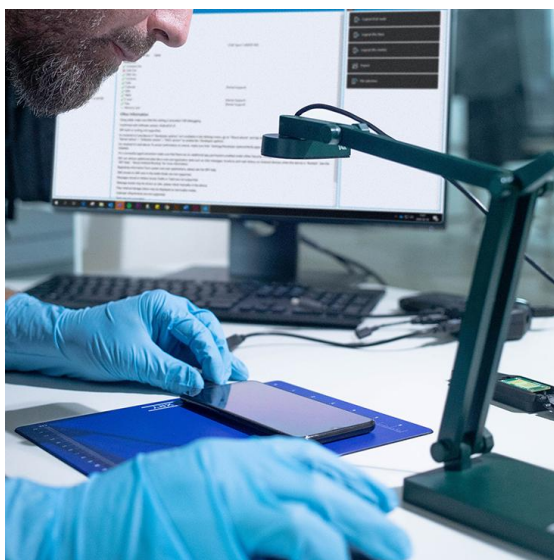
3.2.1 Extracción Manual

Este método debe utilizarse si:

- no tenemos otra opción
- tenemos pocos conocimientos
- ha fallado una extracción física (binaria) o lógica (de sistemas de archivos, archivos, etc.).

Simplemente haremos observación de los datos que podamos ver con **inspección ocular**:

- Usaremos **directamente la interfaz** (teclado, pantalla táctil) para buscar información de forma directa.
- Documentaremos las pruebas haciendo **fotografías**.
- Este proceso es **rápido y fácil** y funciona en prácticamente todos los dispositivos.
- Puede ser que **cometamos errores** al hacerlo si no estamos familiarizados con la interfaz.
- No permite recuperar información borrada.
- Podemos **destruir datos o simplemente cambiarlos sin darnos cuenta** (por ejemplo, si miramos un SMS, se quedará marcado como leído).



<https://www.msab.com/product/xry-extract/xry-camera/>

3.2.2 Extracción Lógica

Se trata de **conectar el dispositivo vía cable USB, RJ45, infrarrojos o Bluetooth**.

- Una vez conectado se **comandos al dispositivo** para que los interprete y devuelva los datos solicitados a la estación forense.
- Más tarde se revisan los datos.
- Es un **proceso rápido y fácil de usar** y requiere poco entrenamiento.
- Se puede escribir sin querer en el dispositivo y cambiar su integridad.
- Normalmente **tampoco podemos recuperar información borrada**.

La gran mayoría de herramientas están a este nivel.

3.2.3 Extracción de volcado binario (Hex Dump)

Se trata de una extracción que se conoce como **física del contenido de la memoria en crudo**.

- Se conecta el dispositivo a la estación forense y se inserta en el dispositivo un **código de arranque no firmado** o bien un **bootloader**.
- Con estas herramientas se hace el vuelco de memoria.
- Requiere **pericia técnica para analizar la imagen binaria**.
- El proceso es poco costoso, permite obtener más información y permite recuperar datos borrados del espacio no asignado.
- **No siempre es factible**
 - Requiere acceso completo al almacenamiento y no siempre está disponible (**hay que ser root**)
 - En muchas ocasiones **se necesita usar exploits** para saltar la seguridad y poder acceder.

3.2.4 Lectura de memoria física sin extracción

También se puede utilizar el método **Joint Test Action Group (JTAG)** que permite leer la memoria sin extraerla.

- Se trata de conectar a los **Test Access Ports (TAPs)** del dispositivo para forzar la transferencia de datos en crudo almacenados en el dispositivo.
- Se utiliza en casos en que **el dispositivo funciona**, pero no es accesible por los métodos anteriores.

Estas dos técnicas (chip-off y JTAG) también se aplican **cuando el dispositivo tiene la pantalla en bloqueo y no es posible saltarla**.

3.2.5 Extracción del chip (Chip-off)

Se trata de **extraer la información directamente del chip del dispositivo**.

- Tenemos memorias flash:
 - NAND, la más típica
 - NOR, que optimiza la ejecución de código, pero no se utiliza tanto.
- Se **extrae físicamente la memoria** para ponerla a un lector de memorias u otro teléfono.
- El proceso es caro, requiere conocimientos del hardware y utilizar herramientas para desoldar y calentar el chip.
- **Es necesario un buen entrenamiento** para hacerlo correctamente.
- Este método se prefiere en el caso de que sea muy importante preservar el estado de la memoria de forma exacta.

Peligros:

- Una mala ejecución puede estropear el chip de memoria.
- Sólo **se recomienda si las opciones anteriores han fallado**, ya que es un proceso destructivo.
- La **información que se obtiene está en formato crudo** y hacen falta herramientas para decodificarla e interpretarla

- Es la única alternativa **si el dispositivo no funciona pero el chip está intacto.**



Chip off: <https://www.teeltech.com/mobile-device-forensics-training/combined-jtag-chipoff-forensics-training/>

3.2.6 Micro Lectura

Se trata de **leer las señales electrónicas generadas por las patas del chip**, por ejemplo con osciloscopios de precisión.

- Es un proceso laborioso, costoso, que necesita mucho tiempo y muchos conocimientos técnicos.
- Sólo para casos de mucha importancia, a nivel de seguridad nacional.
- Todas las herramientas son comerciales.

4 Evidencias potenciales en los dispositivos móviles

Se pueden encontrar **evidencias en diferentes localizaciones:**

- tarjetas **SIM**
- tarjetas de memoria externa (**SD, microSD**)
- memoria **RAM**
- memoria **FLASH interna**
- **proveedor de servicio telefónico**
- **nube**

En general se pueden obtener los siguientes artefactos, la mayoría con marcas de tiempo:

- **Libro de direcciones (Address book):** Nombres de contacto, número de teléfono, emails...
- **Histórico de llamadas (Call history):** Llamadas hechas, recibidas y perdidas así como su duración.
- **SMS:** Mensajes de texto enviados y recibidos.
- **MMS:** Archivos como fotos enviadas y recibidas o fotos por MMS.
- **E-mail:** emails enviados, en borrador y recibos.
- **Histórico del navegador Web:** páginas web visitadas.

- **Fotos:** Hechas por el móvil, descargadas de internet o transferidas de otro dispositivo.
- **Videos:** Capturados con la cámara, descargados de Internet, transferidos de otros dispositivos.
- **Música:** Ficheros de música de Internet u otros dispositivos.
- **Documentos:** Creados con aplicaciones del dispositivo, bajados de Internet, o transferidos.
- **Calendario:** Citas y tareas.
- **Comunicaciones de redes:** Wifi, Bluetooth, etc.
- **Mapas y Ubicaciones GPS:** Lugares visitados, direcciones que se han buscado, mapas descargados...
- **Datos de redes sociales:** Datos guardados por aplicaciones como Facebook, Twitter, LinkedIn, WhatsApp, etc.
- **Datos borrados:** información borrada del teléfono.

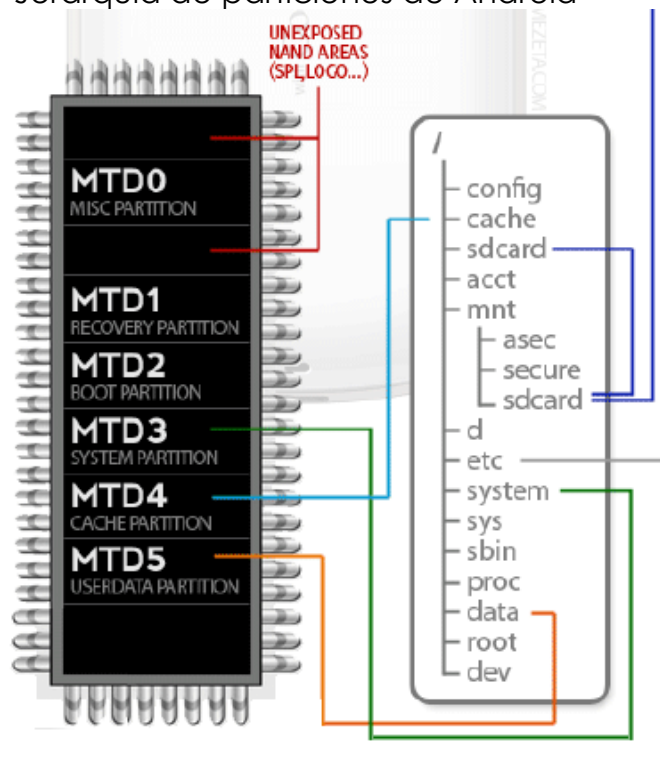
5 Particiones del sistema Android

Como Android utiliza Linux como base, la jerarquía de directorios es muy parecida.

- Cada fabricante puede introducir cambios poco importantes en esta jerarquía de directorios.
- **Algunos directorios solo los puede acceder el usuario root** y por lo tanto se necesitará hacer root en el dispositivo.



Jerarquía de particiones de Android



Jerarquía de particiones vs directorios Android

La estructura de directorios se puede ver con el pedido "ls", aunque en un dispositivo donde no se es root no se puede acceder a muchos directorios:

```
rodin:/ $ ls -la
total 96
drwxr-xr-x 31 root root 755 2009-01-01 01:00 .
drwxr-xr-x 31 root root 755 2009-01-01 01:00 ..
lrw-r--r-- 1 root root 30 2009-01-01 01:00 adb_keys -> /product/etc/security/adb_keys
drwxr-xr-x 83 root root 1680 2026-03-06 12:25 apex
lrw-r--r-- 1 root root 11 2009-01-01 01:00 bin -> /system/bin
drwxr-xr-x 10 root root 220 2026-02-03 12:09 bootstrap-apex
lrw-r--r-- 1 root root 50 2009-01-01 01:00 bugreports -> /data/user_de/0/com.android.shell/files/bugreports
l???????? ? ? ? ? ? cache -> ?
drwxr-xr-x 3 root root 0 1970-01-01 01:00 config
drwxr-xr-x 5 root root 108 2025-07-03 08:18 cust
lrw-r--r-- 1 root root 17 2009-01-01 01:00 d -> /sys/kernel/debug
drwxrwx--x 68 system system 4096 2026-02-03 12:10 data
d???????? ? ? ? ? ? data_mirror
drwxr-xr-x 2 root root 27 2009-01-01 01:00 debug_ramdisk
drwxr-xr-x 24 root root 9420 2026-03-08 23:33 dev
lrw-r--r-- 1 root root 11 2009-01-01 01:00 etc -> /system/etc
l???????? ? ? ? ? ? init -> ?
-???????? ? ? ? ? ? init.environ.rc
d???????? ? ? ? ? ? linkerconfig
d???????? ? ? ? ? ? metadata
drwxr-xr-x 6 root root 101 2009-01-01 01:00 mi_ext
drwxr-xr-x 20 root system 420 2026-02-03 12:10 mnt
drwxr-xr-x 10 root root 159 2009-01-01 01:00 odm
drwxr-xr-x 3 root root 42 2009-01-01 01:00 odm_dtkm
drwxr-xr-x 2 root root 27 2009-01-01 01:00 oem
drwxr-xr-x 2 root root 27 2009-01-01 01:00 opconfig
drwxr-xr-x 2 root root 27 2009-01-01 01:00 opcust
d???????? ? ? ? ? ? postinstall
dr-xr-xr-x 1260 root root 0 1970-01-01 01:00 proc
drwxr-xr-x 17 root root 289 2009-01-01 01:00 product
lrw-r--r-- 1 root root 21 2009-01-01 01:00 sdcard -> /storage/self/primary
drwxr-xr-x 2 root root 27 2009-01-01 01:00 second_stage_resources
drwx--x--- 4 shell everybody 100 2026-02-03 12:09 storage
dr-xr-xr-x 21 root root 0 1970-01-01 01:00 sys
drwxr-xr-x 13 root root 345 2009-01-01 01:00 system
d???????? ? ? ? ? ? system_dtkm
drwxr-xr-x 13 root root 207 2009-01-01 01:00 system_ext
drwxrwx--x 2 shell shell 40 2026-02-03 12:09 tmp
drwxr-xr-x 15 root shell 323 2009-01-01 01:00 vendor
drwxr-xr-x 4 root root 57 2009-01-01 01:00 vendor_dtkm
```

Los dispositivos Android tienen **3 particiones muy importantes**: Boot, System y Recovery.

- **/boot (Boot loader)**: Es la primera partición en cargar.
 - Decide cuál será la siguiente partición a iniciarse, que normalmente es la ROM de Android.
 - Contiene el kernel y la ramdisk (memoria virtual que ejecuta algunos procesos iniciales).
 - Se utiliza también cuando se hace un **reset de fábrica** del dispositivo y para instalar actualizaciones.

- **/system (Android ROM):** Contiene el sistema operativo Android.
 - También contiene las aplicaciones del sistema.
 - Nunca se debe borrar por que entonces no se podría iniciar el teléfono.
 - Es de sólo lectura.

```
rodin:/ $ cd system
rodin:/system $ ls -la
total 140
drwxr-xr-x 13 root root    345 2009-01-01 01:00 .
drwxr-xr-x 31 root root    755 2009-01-01 01:00 ..
drwxr-xr-x  2 root root   1774 2009-01-01 01:00 apex
drwxr-xr-x  1 root root    27 2009-01-01 01:00 app
drwxr-x--x  4 root shell 8644 2009-01-01 01:00 bin
-rw-----  1 root root   9882 2009-01-01 01:00 build.prop
drwxr-xr-x 19 root root   1841 2009-01-01 01:00 etc
drwxr-xr-x  2 root root   8590 2009-01-01 01:00 fonts
drwxr-xr-x  1 root root    99 2009-01-01 01:00 framework
drwxr-xr-x  4 root root  23953 2009-01-01 01:00 lib
drwxr-xr-x  5 root root 26009 2009-01-01 01:00 lib64
lrw-r--r--  1 root root    14 2009-01-01 01:00 media -> /product/media
drwxr-xr-x  1 root root    27 2009-01-01 01:00 priv-app
lrw-r--r--  1 root root     8 2009-01-01 01:00 product -> /product
-rw-r--r--  1 root root   186 2009-01-01 01:00 ro.prop
-rw-r--r--  1 root root     0 2009-01-01 01:00 rw.prop
lrw-r--r--  1 root root    11 2009-01-01 01:00 system_ext -> /system_ext
drwxr-xr-x  7 root root   138 2009-01-01 01:00 usr
lrw-r--r--  1 root root     7 2009-01-01 01:00 vendor -> /vendor
drwxr-x---  2 root shell    56 2009-01-01 01:00 xbin
```

- **/recovery:** Partición de recuperación que contiene software obviamente de recuperación.
 - El menú de Recovery tiene **menos opciones que el Fastboot**
 - Para ello hay software de recuperación hecho a medida (custom recovery).

```
emu64xa:/ # find . -name "recovery" 2> /dev/null
./data/misc/recovery
./data/cache/recovery
```

Hay algunas particiones más, interesantes para el forense:

- **/data (Datos):** Contiene los datos de los usuarios (contactos, sms, llamadas...).
 - Son las aplicaciones que el usuario instala.
 - Es lo que se conoce como la memoria interna del teléfono.
 - Esta partición se reinicializa al hacer el proceso de "parámetros de fábrica".
 - Sólo accesible si somos root.
- ***/cache:** Contiene datos y archivos accedidos a menudo por parte del usuario y el sistema.
 - Sirve para mejorar el rendimiento del teléfono.
 - Es un área de almacenamiento temporal.
 - Puede contener datos que ya no existen en la carpeta /fecha

```
rodin:/ $ find / -name "cache" 2> /dev/null
/storage/emulated/0/Android/data/com.microsoft.appmanager/cache
/storage/emulated/0/Android/data/com.miui.cloudservice/cache
/storage/emulated/0/Android/data/com.google.android.gms/cache
```

- ***/misc:** Guarda configuraciones del operador y hardware del teléfono.
 - Importante para el funcionamiento correcto del teléfono.
 - Contiene una serie de variables On/Off que indican estados del dispositivo, USB, etc.

```
1|rodin:/ $ find / -name "misc" 2> /dev/null
/dev/block/by-name/misc
/dev/block/platform/soc/112b0000.ufshci/by-name/misc
/sys/class/misc
/sys/devices/platform/soc/13000000.mali/misc
/sys/devices/platform/soc/1c00b000.watchdog/misc
/sys/devices/platform/soc/11013800.spi3/spi_master/spi3/spi3.0/misc
/sys/devices/virtual/misc
```

Otros directorios son:

- **/proc:** Con información de los procesos en ejecución
- **/mnt:** punto de montaje para otros tipos de almacenamiento.
- **/sdcard:** Espacio para guardar ficheros en las tarjetas SD.
 - Se crea una carpeta para cada SD.
 - Los nombres son sdcard0, sdcard1...

```
1|rodin:/ $ ls -la sdcard/
total 127
drwxrws--- 2 u0_a282 media_rw 3452 2025-09-13 22:10 Alarms
drwxrws--x 5 media_rw media_rw 3452 2025-09-13 22:09 Android
drwxrws--- 2 u0_a282 media_rw 3452 2025-09-13 22:10 Audiobooks
drwxrws--- 4 u0_a282 media_rw 3452 2025-11-09 22:48 DCIM
drwxrws--- 2 u0_a282 media_rw 3452 2026-03-04 21:53 Documents
drwxrws--- 14 u0_a282 media_rw 118784 2026-03-04 22:11 Download
drwxrws--x 2 media_rw media_rw 3452 2025-09-13 22:09 HyperOS_Sandbox
drwxrws--- 7 u0_a282 media_rw 3452 2025-11-09 23:45 MIUI
drwxrws--- 3 u0_a282 media_rw 3452 2025-09-13 22:10 Movies
drwxrws--- 3 u0_a282 media_rw 3452 2025-09-13 22:10 Music
drwxrws--- 2 u0_a282 media_rw 3452 2025-09-13 22:10 Notifications
drwxrws--- 5 u0_a282 media_rw 3452 2026-02-11 07:19 Pictures
drwxrws--- 2 u0_a282 media_rw 3452 2025-09-13 22:10 Recordings
drwxrws--- 2 u0_a282 media_rw 3452 2025-09-13 22:10 Ringtones
```

-
- **/sd-ext:** partición adicional a sdcard, en algunas ROMs customizadas.
- **/efs:** carpeta con IMEI, PRODUCTCODE, SIM, UNLOCK ...
- **/data/data:** Uno de los directorios más importantes ya que contiene los datos privados de las aplicaciones.
 - /data/data/<nombre_paquete>
 - Contiene subdirectorios importantes como:
 - shared_prefs: ficheros XML con configuración de las Apps
 - databases: bases de datos de las App en formato SQLite.

Para ver las particiones del sistema Android, podemos ejecutar el comando "df" desde ADB.

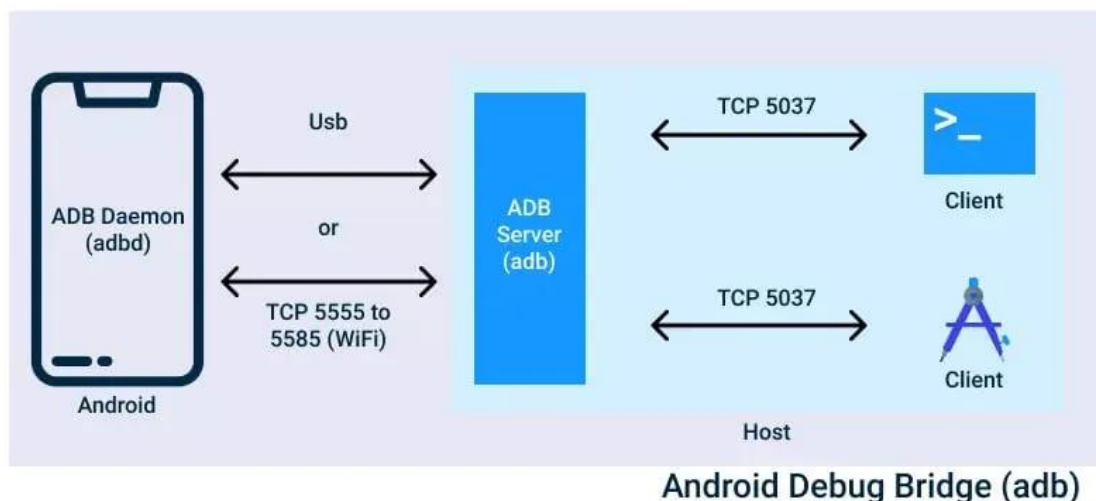
6 ADB

Android Debug Bridge (adb) es una herramienta de línea de comandos versátil que permite comunicarse con un dispositivo Android.

El comando adb permite realizar una gran variedad de acciones en el dispositivo como:

- instalar y depurar apps
- proporcionar acceso a un shell de Unix que se puede usar para ejecutar diferentes comandos en un dispositivo.
- También se puede ejecutar adb desde un navegador Chrome con Webadb - <https://webadb.com/>
- ADB es un programa cliente-servidor que incluye tres componentes:
 - **Un cliente**, que envía comandos y se ejecuta en la máquina de desarrollo (pedido adb).
 - **Un servidor**, que administra la comunicación entre el cliente y el daemon, que se ejecuta en la máquina de desarrollo .
 - **Un daemon (adbd)**, que ejecuta pedidos en el dispositivo Android.

How does it work?



Información en: <https://developer.android.com/studio/command-line/adb?hl=es-419>

6.1 Instalar y utilizar ADB

Instala el Kit de Desarrollo de Software (SDK) para Android.

- Éste contiene:
 - el IDE de desarrollo de apps Android
 - documentación
 - simuladores de Android
 - otras herramientas útiles para la investigación.
- Si tenemos suficiente espacio usaremos esta opción (en Windows o mejor en Linux, ya que Android usa Linux). <https://developer.android.com/studio/index.html>
- En Linux podemos acelerar la emulación de Android con KVM, un hypervisor que reside en el kernel de Linux - https://developer.android.com/studio/run/emulator-acceleration?utm_source=android-studio#vm-linux

- Alternativamente se puede instalar sólo las "**Platform tools**", que forman parte del SDK para ocupar menos espacio. Incluyen por ejemplo ADB.
<https://developer.android.com/tools/releases/platform-tools>

Si quieres trabajar con un emulador y tu equipo tiene poco espacio (Android Studio suele ocupar algo más de 10Gb), es posible instalar solo el emulador junto con ADB así:

1. Descarga **Command line tools for Windows** desde la web oficial de Android desde <https://developer.android.com/studio#command-tools> (están en la parte de abajo)
2. Descomprime el fichero descargado en una carpeta llamada Android. Crea la carpeta **latest** y copia en ella todo el contenido. Debe quedar así:

```

C:\Android
├── cmdline-tools
│   └── latest
│       ├── bin
│       ├── lib
│       └── source.properties

```

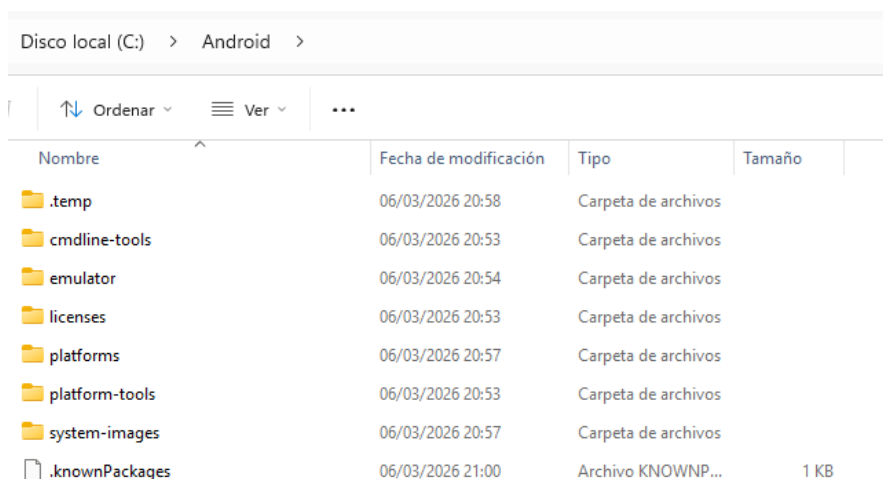
3. Abre PowerShell en la carpeta C:\Android\cmdline-tools\latest\bin y lanza estos comandos para instalar **ADB, Android Emulator, Android 14** y **imagen del sistema Android**:

```

.\sdkmanager.bat "platform-tools"
.\sdkmanager.bat "emulator"
.\sdkmanager.bat "platforms;android-34"
.\sdkmanager.bat "system-images;android-34;google_apis;x86_64"

```

Estos comandos generarán carpetas extra dentro de la carpeta C:\Android



Nombre	Fecha de modificación	Tipo	Tamaño
.temp	06/03/2026 20:58	Carpeta de archivos	
cmdline-tools	06/03/2026 20:53	Carpeta de archivos	
emulator	06/03/2026 20:54	Carpeta de archivos	
licenses	06/03/2026 20:53	Carpeta de archivos	
platforms	06/03/2026 20:57	Carpeta de archivos	
platform-tools	06/03/2026 20:53	Carpeta de archivos	
system-images	06/03/2026 20:57	Carpeta de archivos	
.knownPackages	06/03/2026 21:00	Archivo KNOWNP...	1 KB

4. Es posible que para instalar ADB (platform-tools) indique que necesita java 17 o superior:

```

PS C:\Android\cmdline-tools\latest\bin> .\sdkmanager.bat "platform-tools"
Java version 17 or higher is required.
To override this check set SKIP_JDK_VERSION_CHECK

```

5. Puedes instalar Java y configurar correctamente la variable de entorno "JAVA_HOME" o saltar esta comprobación usando la opción SKIP_JDK_VERSION_CHECK

```
PS C:\Android\cmdline-tools\latest\bin> $env:SKIP_JDK_VERSION_CHECK="true"
PS C:\Android\cmdline-tools\latest\bin> .\sdkmanager.bat "platform-tools"
```

6. Instalamos la plataforma (también podemos elegir la 32) y descargamos la imagen.

```
PS C:\Android\cmdline-tools\latest\bin> .\sdkmanager.bat "platforms;android-34"
PS C:\Android\cmdline-tools\latest\bin> .\sdkmanager.bat "system-images;android-34;google_apis;x86_64"
```

7. Una vez terminada la instalación, puedes crear un emulador Android así:

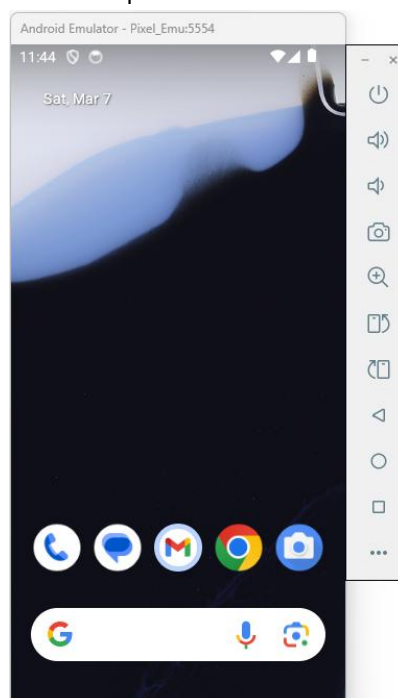
```
PS C:\Android\cmdline-tools\latest\bin> .\avdmanager.bat create avd -n Pixel_Emu -k "system-images;android-34;google_apis;x86_64"
Auto-selecting single ABI x86_64 ] 25% Loading local repository...
Do you wish to create a custom hardware profile? [no]
```

Durante la creación del emulador preguntará si quieres crear un perfil de hardware personalizado, pulsa **intro** y creará un dispositivo Google Pixel

8. Cambia a la carpeta **C:\Android\emulator** y lanza el emulador:

```
PS C:\Android\emulator> .\emulator.exe -avd Pixel_Emu
```

9. Deberá aparecer el emulador en la pantalla:



Nota: Si tienes problemas para lanzar el emulador, instala java, crear la variable de entorno JAVA_HOME cuyo valor debe ser la ruta al fichero de instalación de Java y reinicia toda la instalación (borra la carpeta creada).

Si necesitas eliminar un emulador creado puedes usar el comando **avdmanager.bat delete**:

```
PS C:\Android\cmdline-tools\latest\bin> .\avdmanager.bat delete avd -n Pixel_Emu
```

Puedes listar los emuladores creados con:

```
PS C:\Android\cmdline-tools\latest\bin> .\avdmanager.bat list avd
Available Android Virtual Devices:
  Name: Pixel_Emu
  Path: C:\Users\Juanjo\.android\avd\Pixel_Emu.avd
  Target: Google Play (Google Inc.)
    Based on: Android 14.0 ("UpsideDownCake") Tag/ABI: google_apis_playstore/x86_64
  Sdcard: 512 MB

The following Android Virtual Devices could not be loaded:
  Name: Medium_Phone_API_36.1
  Path: C:\Users\Juanjo\.android\avd\Medium_Phone.avd
  Error: Missing system image for Google Play x86_64 Medium Phone API 36.1.
```

Conectarse a un móvil físico o al emulador

1. Hay que activar antes en el propio móvil:

- **Opciones de desarrollador:** "Configuración" > "Acerca del dispositivo" > "Número de compilación" y picar encima 7 veces o hasta que aparezca el mensaje de que ya somos desarrolladores.
- **Depuración por USB:** "Configuración" > "Sistema" > "Avanzado" > "Opciones para desarrolladores" > "Depuración por USB".

En el emulador no es necesario hacer estos pasos.

2. En algunos casos, será necesario también **instalar los drivers USB para el móvil** utilizado. Por ejemplo: Universal ADB Drivers - <https://adb.clockworkmod.com/>
3. **Conectamos el cable USB o configuramos la conexión Wifi** (wifi por Android 11 y superiores).
 - Configurar debug wifi:
<https://desarrollador-android.com/desarrollo/herramientas/herramientas-de-ayuda/depurar-aplicaciones-via-wifi/>
4. El móvil físico nos pedirá autorización para conectarse a él desde el pc. Nos mostrará la huella digital del dispositivo que se conecta.

Detalles en manual de ADB: <https://developer.android.com/studio/debug/dev-options?hl=es-419#enable>

6.2 Comandos ADB

Con ADB tenemos **control del móvil desde el pc**. Podemos ejecutar muchos comandos, entre ellos:

Conexión y control

- `adb connect <IP:PORT>`: Conecta a un dispositivo
- `adb devices -l`: Para ver dispositivos conectados
- `adb kill-server`: Detiene el servidor adb en caso de problemas (cualquier comando adb lo vuelve a poner en marcha).
- `adb -s [NUMERO-DISP] shell comando`: para ejecutar comando y salir. Si no se pone comando, se abre un shell interactivo con el dispositivo.
- `adb shell`: Abre un shell interactivo con el dispositivo. Se podrán escribir comandos del Linux de Android

```
PS C:\Android\platform-tools> .\adb.exe shell
emu64xa:/ $ ls
acct      cache      debug_ramdisk  linkerconfig  odm_dlkmsdcard      system_dlkmsystem_ext
adb_keys  config     dev            lost+found    oem          second_stage_resources
apex      d          etc            metadata      postinstall  storage          vendor
bin       data       init           mnt           proc         sys              vendor_dlkmsystem
bugreports data_mirror init.environ.rc odm            product      system
```

Si tenemos más de un dispositivo conectado (móvil físico y emulador):

```
PS C:\Android\platform-tools> .\adb.exe devices -l
List of devices attached
DENFMR8DCQ6HX4EY    device product:rodin_eea model:2412DPC0AG device:rodin transport_id:7
emulator-5554        device product:sdk_gphone64_x86_64 model:sdk_gphone64_x86_64 device:emu64xa transport_id:6

PS C:\Android\platform-tools> .\adb.exe -s DENFMR8DCQ6HX4EY shell
rodin:/ $ ls
adb_keys  config     dev            mi_ext  opcust      storage      vendor
apex      cust       etc            mnt     postinstall  sys          vendor_dlkmsystem
bin       d          init           odm     proc         system
bootstrap-apex data       init.environ.rc odm_dlkmsproduct  system_dlkmsystem_ext
bugreports data_mirror linkerconfig  oem     sdcard      system_ext
cache     debug_ramdisk metadata      opconfig second_stage_resources tmp
```

- `adb reboot`: Reiniciar dispositivo.
- `adb reboot-bootloader`: Reiniciar dispositivo en modo fastboot.
- `adb reboot recovery`: Reiniciar dispositivo en modo recuperación.
- `adb root`: Eleva a privilegios de root (sólo en entornos de desarrollo, no de producción)

```
PS C:\Android\platform-tools> .\adb.exe root
restarting adbd as root
PS C:\Android\platform-tools> .\adb.exe shell
emu64xa:/ # ls
acct      cache      debug_ramdisk  linkerconfig  odm_dlkmsdcard      system_dlkmsystem_ext
adb_keys  config     dev            lost+found    oem          second_stage_resources
apex      d          etc            metadata      postinstall  storage          vendor
bin       data       init           mnt           proc         sys              vendor_dlkmsystem
bugreports data_mirror init.environ.rc odm            product      system
```


Información del sistema

- `adb logcat`: Logs del sistema Android (seguimiento de pila, errores del sistema y mensajes que se envían desde una app con la clase Log)
- `adb shell dumpsys`: Volcar datos del sistema
- `adb shell dumpsys | grep "DUMP OF SERVICE servicio"`: Volcar datos del sistema a pantalla centrado en un servicio
- `adb shell dumpsys [servicio]`: Volcar datos de un servicio específico
- `adb shell dumpsys -l`: Muestra una lista con los posibles servicios a usar con `dumpsys`
- `adb shell cat /proc/cpuinfo`: Mostrar información sobre CPU

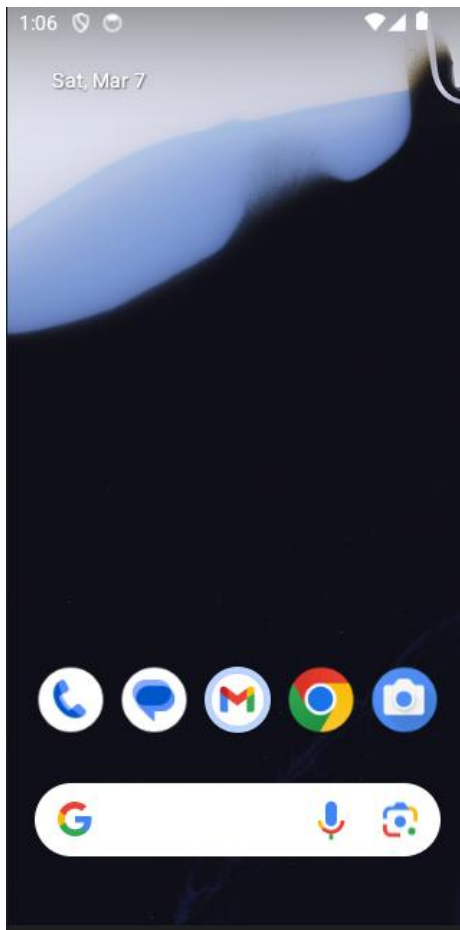
Enviar y recibir ficheros

- `adb push [ruta-local] [ruta-dispositivo]`: Copiar un archivo en el dispositivo
- `adb pull [ruta-dispositivo] [ruta-local]`: Descargar un archivo desde el dispositivo
 - Ejemplo: `adb pull /data/misc/wifi/wpa_supplicant.conf ./info`
fichero que contiene las wifis y passwords de las wifis accedidas
- `adb backup`: Google dice que está "deprecated" pero de momento se puede usar y es una de las mejores opciones. Sólo sirve para las aplicaciones configuradas con el permiso de backup sin ser root.
 - Ejemplo: `adb backup -apk org.fedorahosted.freeotp -f freeotp.adb`
- (02-2024) Copia de seguridad de la app Android, datos incluidos, sin necesidad de root, con adb -
<https://gist.github.com/AnatomicJC/e773dd55ae60ab0b2d6dd2351eb977c1>

Capturar pantalla

- `adb shell screencap <ruta-dispositivo-fichero-captura>`: Captura de pantalla
 - Ejemplo: Capturar pantalla sin tocar físicamente la pantalla del móvil (También se pueden grabar vídeos de la pantalla):

```
PS C:\Android\platform-tools> .\adb.exe shell
emu64xa:/ # screencap /sdcard/screen.png
emu64xa:/ # exit
PS C:\Android\platform-tools> .\adb.exe pull /sdcard/screen.png
/sdcard/screen.png: 1 file pulled, 0 skipped. 22.3 MB/s (205116 bytes in 0.009s)
PS C:\Android\platform-tools> |
```



- `adb shell screenrecord -time-limit [según] [ruta-dispositivo]:`
Grabar vídeo de la pantalla del dispositivo

Simular pulsaciones de botones

- `adb shell input keyevent 26:` Emular botón encender
- `adb shell input keyevent 82:` Emular pantalla de desbloqueo

APK's

- `adb install <fichero.apk>:` Instalar APK
- `adb -r install <fichero.apk>:` Reinstalar APK (no borra datos)
- `adb uninstall <nombre-paquete-aplicación>:` Desinstalar APK
- `adb shell pm path <nom-paquete>:` Extraer APK
- `adb shell pm list packages:` Listar paquetes instalados
 - Ejemplo: Listar paquetes instalados por el fabricante oneplus
 - Paquetes de listas pm | Grep 'OnePlus'
- `adb shell pm uninstall -k --user 0 NameOfPackage:` Borrar aplicación, útil para eliminar "Bloatware" (software preinstalado por el fabricante).
 - `adb shell pm desinstalar -k --user 0 com.mi.android.globalFileexplorer`

<https://www.xda-developers.com/uninstall-carrier-oem-bloatware-without-root-access/>

- `adb shell am start -n [paquete-aplicación]/. [activity] :` Iniciar una Activity (apk).

Downgrade de aplicaciones

Hay aplicaciones que están instaladas previamente en el dispositivo que pueden evitar la extracción de los datos con un proceso de backup:

- El método de backup es uno de los métodos que se utilizan por los forenses para extraer los datos del dispositivo.
- El método de "**downgrade**" consiste en instalar una versión anterior del programa manteniendo los datos que contenía.
- La práctica de downgrade **permite acceder a vulnerabilidades de versiones anteriores de software que actualmente estén arregladas**, por ejemplo, Whatsapp. Así podremos extraer los datos.

<https://www.xda-developers.com/downgrade-an-app-android-no-root/>

```
adb push app.apk /sdcard/app.apk
// Retener datos -d downgrade
adb shell pm install -r -r /sdcard/app.apk
```

Herramientas para hacer downgradadas - AvillaForensics –

<https://github.com/AvillaDaniel/AvillaForensics>

Activar datos y localización

- `adb shell svc data enable`: Habilitar datos móviles
- `adb shell svc data disable`: Deshabilitar datos móviles
- `adb shell location`: Localizar dispositivo

6.3 Duplicar pantalla del dispositivo en el equipo forense

Si queremos trabajar más cómodo o necesitamos hacer alguna demostración, podemos **ver la pantalla de nuestro móvil Android físico en la pantalla**. Podemos utilizar entre otras las aplicaciones:

- **Scrcpy** - Permite conexión vía USB o sobre TCP/IP y Wifi, con muy poca latencia. Permite usar ratón y teclado del PC para controlar el móvil. No requiere Root
 - Manual y repositorio de fuentes: <https://github.com/Genymobile/scrcpy>
 - Otros repos con herramientas para Android - <https://github.com/rom1v>
 - Blog personal del autor: <https://scrcpy.org/>
 - Si instalas en Ubuntu esta aplicación, ya instala automáticamente adb ya que funciona con la depuración USB.
- **Screenstream**: Al Google Play.
- **TeamViewer**: Administrador de dispositivos remotos Android, Windows, Linux...
- **Vysor**: <https://www.vysor.io/>
- **LetsView**: <https://letsview.com/es/>
- **Apower Mirror**: <https://www.apowersoft.es/phone-mirror>
- **Android Mirror**: <https://www.wondershare.com/android-mirror/>

7 Artefactos Android

Al igual que hacemos con los PC's a cada elemento que podemos estudiar o extraer del S.O. también se le denomina artefactos. Se puede consultar una referencia muy completa de los mismos en <https://github.com/RealityNet/Android-Forensics-References>

7.1 SMS/MMS, Call log

Datos:

- **SMS i MMS:** \data\data\com.android.providers.telephony\databases\mmssms.db
- **Registro de llamadas:**
\data\data\com.android.providers.contacts\databases\calllog.db

Valor forense:

- Recuperar el contenido completo de los mensajes de texto.
- Proporcionar números y marcas horarias de remitente/receptor.
- Permite la reconstrucción de la conversación y la validación de la línea de tiempo.
- Útil para mapear patrones de comunicación de usuario.

Tablas de la BD mmssms.db

\datos\datos\com.android.providers.telefonía\databases\mmssms.db

Podemos incluso consultar el contenido desde adb con sqlite3:

```
PS C:\Android\platform-tools> .\adb.exe shell
emu64xa:/ # cd data/data/com.android.providers.telephony/databases
emu64xa:/data/data/com.android.providers.telephony/databases # sqlite3 mmssms.db
SQLite version 3.39.2 2022-07-21 15:24:47
Enter ".help" for usage hints.
sqlite> .table
addr                pdu_restricted      threads
android_metadata    pending_msgs         words
attachments         rate                words_content
canonical_addresses raw                  words_segdir
drm                 sms                  words_segments
part                sms_restricted
pdu                 sr_pending
sqlite> .exit
emu64xa:/data/data/com.android.providers.telephony/databases #
```

7.2 Contactos y Addressbook

Datos:

- \datos\datos\com.android.providers.contactos\databases\contacts2.db

Valor forense:

- Identifica las conexiones sociales.
- Proporciona potenciales enlaces entre números de teléfono y correos electrónicos.
- Permite la triangulación con registros de llamadas y mensajes.

Tablas de contacts2.db

- \data\data\com.android.providers.contacts\databases\contacts2.db

Si intentamos acceder a la carpeta en un dispositivo físico no rooteado:

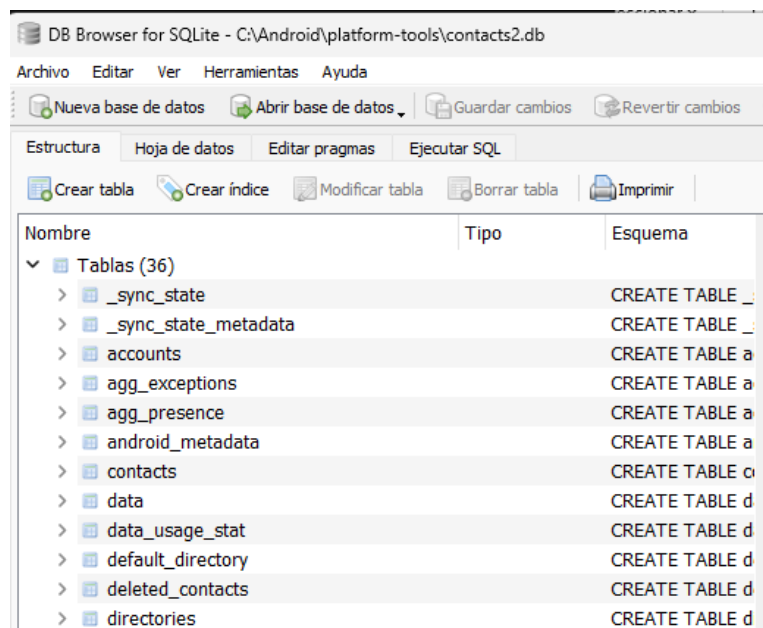
```
PS C:\Android\platform-tools> .\adb.exe -s DENFMR8DCQ6HX4EY shell
rodin:/ $ adb pull /data/data/com.android.providers.contacts/databases/contacts2.db
/system/bin/sh: adb: inaccessible or not found
127|rodin:/ $ exit
PS C:\Android\platform-tools> .\adb.exe devices -l
List of devices attached
DENFMR8DCQ6HX4EY      device product:rodin_eea model:2412DPC0AG device:rodin transport_id:7
emulator-5554         device product:sdk_gphone64_x86_64 model:sdk_gphone64_x86_64 device:emu64xa transport_id:6

PS C:\Android\platform-tools> .\adb.exe -s DENFMR8DCQ6HX4EY shell
rodin:/ $ cd /data/data/com.android.providers.contacts/databases
/system/bin/sh: cd: /data/data/com.android.providers.contacts/databases: Permission denied
2|rodin:/ $
```

En cambio si lo hacemos en el emulador (con adb root habilitado):

```
PS C:\Android\platform-tools> .\adb.exe devices -l
List of devices attached
DENFMR8DCQ6HX4EY      device product:rodin_eea model:2412DPC0AG device:rodin transport_id:7
emulator-5554         device product:sdk_gphone64_x86_64 model:sdk_gphone64_x86_64 device:emu64xa transport_id:6

PS C:\Android\platform-tools> .\adb.exe -s emulator-5554 pull /data/data/com.android.providers.contacts/databases/contacts2.db
/data/data/com.android.providers.contacts/databases/contacts2.db: 1 file pulled, 0 skipped. 25.3 MB/s (352256 bytes in 0.013s)
PS C:\Android\platform-tools>
```



7.3 Historial del navegador

Datos:

- \data\data\com.android.chrome\app_chrome\Default\History

Valor forense:

- Muestra los hábitos de uso de Internet y los intereses de búsqueda
- Puede indicar planificación o intención (p. ej., búsquedas sobre delitos o rutas de escape)
- Puede exponer servicios web accedidos como almacenamiento en nube o foros

Tablas de History

- \data\data\com.android.chrome\app_chrome\Default\History

```
PS C:\Android\platform-tools> .\adb.exe -s emulator-5554 shell
emu64xa:/ # cd /data/data/com.android.chrome/app_chrome/Default
emu64xa:/data/data/com.android.chrome/app_chrome/Default # sqlite3 History
SQLite version 3.39.2 2022-07-21 15:24:47
Enter ".help" for usage hints.
sqlite> .table
cluster_keywords          downloads                 segment_usage
cluster_visit_duplicates downloads_slices         segments
clusters                 downloads_url_chains     typed_url_sync_metadata
clusters_and_visits      history_sync_metadata    urls
content_annotations      keyword_search_terms     visit_source
context_annotations      meta                     visits
sqlite> .exit
```

7.4 Bluetooth

Datos:

- \data\misc\bluedroid\bt_config.xml

```
PS C:\Android\platform-tools> .\adb.exe -s emulator-5554 shell
emu64xa:/ # cat /data/misc/bluedroid/bt_config.conf
[Info]
FileSource = Empty
TimeCreated = 2026-03-06 20:03:24

[Metrics]
Salt256Bit = a607abe393ab8f142af2d8d4c8653c5e5bd1f18afe6b6322bb90b225520d3cdb

[Adapter]
Address = bb:bb:bb:00:00:01
LE_LOCAL_KEY_IRK = c7c63640bb80dfa24fd6cf9026c85032
LE_LOCAL_KEY_IR = 3edc41cbc537e8288bf9403e7c3afdfd
LE_LOCAL_KEY_DHK = 632ac39b1053b69dbf2b875d30387c37
LE_LOCAL_KEY_ER = eefce3ce0369f598ac25073b1330d38a
ScanMode = 1
DiscoveryTimeout = 120
```

7.5 Wifi

Datos:

- /data/misc/wifi/WifiConfigStore.xml
- /data/misc/wifi/wpa_supplicant.conf
- /data/data/com.android.providers.settings: Datos sobre proveedores estándar Android (entre ellos wifi)
- Información sobre el servicio wifi: `adb shell dumpsys wifi`

7.6 GPS

Datos:

- /data/data/com.google.android.gms/databases/ (múltiples ficheros)
- location.db, networklocations.db o com.google.android.location

Valor forense:

- Mapas de movimientos de usuarios y lugares frecuentados.
- Ayuda a confirmar o refutar la presencia en una escena del crimen.
- Crucial en la reconstrucción de actividades y rutas pasadas.

7.7 Fotos y video

Datos:

- /sdcard/DCIM/
- /sdcard/Pictures/
- /sdcard/WhatsApp/Media/

Valor forense:

- Proporciona líneas de tiempo visuales.
- Puede incluir datos GPS y marcas horarias.
- A veces, muestra imágenes suprimidas en miniaturas o previsualizaciones.

7.8 Mensajería Whatsapp

Datos:

- /data/data/com.whatsapp/databases/msgstore.db
- /sdcard/WhatsApp/Media/ (for images, voice notes, etc.)

Valor forense:

- Evidencias directas de conversaciones.
- Altamente relevante en investigaciones penales o civiles.
- Los adjuntos multimedia a menudo sirven como prueba en cualquier caso forense.
- Expone la comunicación a través de canales encriptados y no cifrados

7.9 Datos de aplicaciones

Datos:

- /data/data/[app.package.name]/

Por ejemplo:

- /data/data/com.instagram.android/
- /data/data/com.snapchat.android/

Valor forense:

- Puede descubrir los patrones de uso de la aplicación
- Expone la actividad de fondo no visible en la Interfaz de Usuario.
- Datos ocultos como las credenciales en la memoria cache, los registros o las estadísticas de uso
- Examinar una aplicación sospechosa

7.10 Uso de aplicaciones

Datos:

- /data/system/usagestats/

El comando **adb shell usagestats** nos permite obtener una cronología del uso de cada aplicación (cuando se inició, tiempo de uso, última vez utilizada, ...)

Este tipo de comandos suministran mucha información, con lo que la mejor opción es derivar la salida a un fichero de texto para poder interpretar los datos.

```
PS C:\Android\platform-tools> .\adb.exe -s emulator-5554 shell dumpsys usagestats > datos.txt
```

7.11 Cuentas de usuarios y servicios de Google

Datos:

- /data/system/users/0/accounts.db
- /data/data/com.google.android.gms/databases

Valor forense:

- Identifica identidades de usuario y servicios en línea enlazados
- Muestra servicios utilizados (p. ej., Gmail, Dropbox e Instagram)
- Muestra la frecuencia de sincronización y los vectores potenciales de exfiltración de datos
- Permite la correlación con investigaciones web o basadas en la nube

7.12 Aplicaciones instaladas

Datos:

- /data/system/packages.xml

Valor forense

- Visualizar el uso de la aplicación a lo largo del tiempo
- Permite crear una línea de tiempo o mostrar un patrón de comportamiento
- Evidencia de instalaciones/desinstalaciones de aplicaciones
- Enumera permisos peligrosos que podrían relacionarse con la vigilancia o la explotación.

Con el comando `adb shell dumpsys package` podemos obtener fechas de instalación y actualización de aplicaciones (info en las propiedades **firstInstallTime** y **lastUpdateTime**). Podemos afinar indicando el paquete con `adb shell dumpsys package nombre.paquete`

7.13 Historial de descargas

Datos:

/data/data/com.android.providers.downloads/databases/downloads.db

Valor forense

- Contiene la siguiente información:
 - archivos descargados
 - URL origen
 - Fecha

7.14 Log de Eventos

La utilidad `adb logcat` es muy potente a la hora de extraer información de los eventos que ocurren en un sistema Android.

Ejecutando `adb logcat --help` podemos observar todas sus posibilidades. No obstante, las más interesantes son:

- `adb logcat -b events` → Este buffer contiene **eventos estructurados del sistema**.
- `adb logcat -b events -d > datos.txt` → Se extraen a fichero de texto
- `adb logcat -d > datos.txt` → Log completo a fichero de texto

Dentro del buffer events, los eventos más significativos son los de **Android ActivityManager**:

- El evento **m_activity_resume: com.android.chrome/.Main** indica que la app Chrome pasó a primer plano
- El evento **am_activity_pause: com.whatsapp/.HomeActivity** indica que WhatsApp pasó a segundo plano

Android también registra eventos de aplicaciones a efectos estadísticos de uso de batería en **Android BatteryStats**. Podemos consultarlos con el comando `adb shell dumpsys batterystats`

Más info en <https://developer.android.com/tools/logcat> y <https://developer.android.com/tools/dumpsys>

8 Desbloqueo y acceso a un móvil Android

Muchos usuarios bloquean su dispositivo para tratar de protegerlo. Si queremos hacer la extracción de datos de un móvil, será necesario acceder a ellos. Veremos algunas **técnicas para saltar el bloqueo del dispositivo**.

Hay técnicas que funcionarán según la versión del sistema o no. Habrá que comprobarlo.

8.1 Antes de intentar desbloquear

De todas formas, hay que recordar algunos puntos para evitar tener que aplicar estas técnicas. Hay que comprobar si el dispositivo está desbloqueado en ese momento y tratar de dar el mejor acceso al dispositivo para la adquisición:

1. Activar si es posible la **depuración por USB**.
2. **Desactivar las opciones de suspensión del móvil** para que no se bloquee automáticamente y se detenga la adquisición (o activar opciones "Stay Awake").
3. **Aumentar el tiempo de suspensión** de la pantalla.
4. **Aislar correctamente el dispositivo** para evitar acceso remoto, borrado remoto, ...
 - Las opciones de Google permiten borrar los datos o bloquear el móvil.
 - Existen aplicaciones de terceros que hacen lo mismo.
 - Podemos activar el **modo Avión** si el móvil no está bloqueado, manteniendo botón de apagado.

8.2 Usar Android Device Manager para desbloquear el móvil (Android<v-5)

Muchos móviles Android llevan el servicio de gestión de Android que permite localizar el móvil perdido. También se puede utilizar para desbloquear el dispositivo.

- Hay que conocer las **credenciales de cuenta de Google**.
- Ir a <http://google.com/android/devicemanager> e iniciar sesión.
- Escoger opción "Lock".
- Entrar un password temporal y picar de nuevo Lock.
- Entrar el password temporal en el dispositivo para desbloquear.

En móviles Samsung, también se podía usar el servicio "Find my mobile".

<https://toolbox.iskysoft.com/android-unlock/android-device-manager-unlock.html>

8.3 Saltar el bloqueo de pantalla

Existen diferentes métodos de bloqueo:

1. **Por patrón:** Un dibujo que se hace uniendo puntos a pantalla.
2. **Por PIN:** Código de 4 dígitos
3. **Por password:** Palabra de paso alfanumérica.
4. **Por huella dactilar:** Con el dedo del propietario.
5. **Por control facial:** El móvil hace una foto de la cara del propietario.

Veremos algunos métodos para saltar el bloqueo del móvil. Estos pueden provocar cambios en los dispositivos y no hay garantía de que funcionen.

- Habría que **probar antes en dispositivos de prueba**.
- **Hay que tener autorización** para hacerlo

8.4 Usar ADB para intentar saltar la protección

Conectaremos el móvil por USB y mediante ADB podemos hacer algunos intentos.

Borrar gesture.key

Para Android anterior a la versión 6.0 "Marshmallow", se puede borrar el fichero gesture.key para borrar el patrón del dispositivo. NOTA: Sólo se puede hacer si el dispositivo está "Rooteado"

- Conectar vía usb
- Ejecutar:

```
adb shell
cd /data/system
rm gesture.key
```
- Reiniciar el dispositivo. Si todavía aparece el bloqueo de patrón, podemos dibujar un aleatoriamente y desbloquear.

Actualizar Settings.db

Se puede buscar la BD settings.db, en formato SQLite, y cambiar los valores de bloqueo a 0.

```
adb shell
cd /data/data/com.android.providers.settings/databases
sqlite3 settings.db
update system set value=0 where name='lock_pattern_autolock';
update system set value=0 where
name='lock_screen.lockedoutpermanently';
.quit
```

Borrar archivos de bloqueo

Se puede intentar borrar los archivos que configuran el bloqueo.

```
adb shell su rm /Data/System/locksettings.db
rm /Data/System/locksettings.db-WAL
rm /data/system/locksettings.db-shm
```

Versiones superiores a Android 6.0

Desde el Android 6.0 "Marshmallow", ya no se guarda la información sobre el método de bloqueo de pantalla en el fichero gesture.key ni password.key. En su lugar se guarda en otros ficheros que crea el gatekeeper-service. Hay que borrar los ficheros:

```
adb Shell rm /data/system/gatekeeper.password.key
adb Shell rm /data/system/gatekeeper.pattern.key
adb Shell rm /Data/System/locksettings.db
```

A partir del Android 4.2 se introdujo el sistema de múltiples usuarios, de manera que los **datos se segmentan para cada usuario "humano"**. Por lo tanto los datos se encuentran en /data/user/0, /data/user/1, /data/user/2, ...

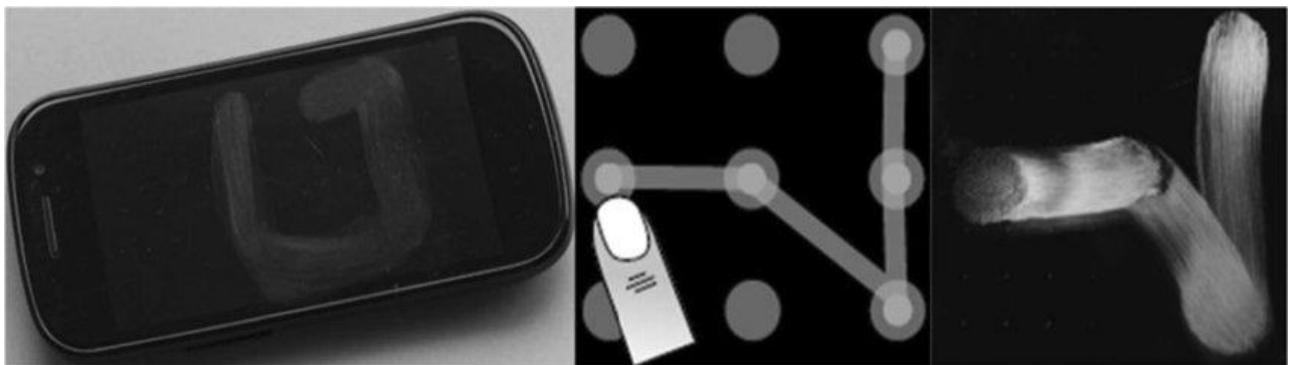
Por retrocompatibilidad la carpeta /data/data tiene un enlace simbólico a /data/user/0.

La BD de datos de configuración se encuentra en /data/user/0/com.android.providers.settings/databases

8.5 Smudge Attack

Smudge: Se trata de las marcas que se dejan en la pantalla al tocarla con los dedos sucios o húmedos. Con una luz adecuada se puede ver el rastro dejado por los dedos, como se puede tembir en la foto siguiente.

- Se puede llegar a **deducir el patrón**.
- Se puede llegar a deducir el PIN del dispositivo (sabiendo dónde están situados los números).



Pantalla con marcas de los dedos (smudge)

8.6 Contraseña o patrón olvidado

Hasta Android 4.4 funcionaba la opción de recuperar password o patrón en la pantalla de desbloqueo.

8.7 Saltar bloqueo de aplicación de terceros

En el caso de que el bloqueo se haga con aplicaciones de terceros, es posible que se pueda saltar arrancando el dispositivo en modo seguro.

- Este modo no carga ninguna de las aplicaciones que se han instalado por el usuario, de manera que se podrá desactivar la opción de que se arranque y bloquee.
- Puede ser que funcione **mantener el botón de power hasta que aparezca el mensaje de reiniciar**. Entonces **mantener el botón power otra vez hasta que aparezca la opción de reiniciar en modo seguro**.
- En **Xiaomi por ejemplo**:
 - se debe apagar el móvil
 - volver a encender
 - cuando aparezca el logo de MIUI mantener el botón de bajar volumen hasta que arranque.
 - Aparecerá un mensaje en la parte inferior izquierda de la pantalla.

8.8 Usar una APK con permiso DISABLE_KEYGUARD

Otra posibilidad es encontrar y usar aplicación con el permiso DISABLE_KEYGUARD (hoy en día bastante difícil).

- Detalle del permiso:
https://developer.android.com/reference/android/Manifest.permission#DISABLE_KEYGUARD
- Posible código de la aplicación:
<https://stackoverflow.com/questions/21852203/android-how-to-completely-disable-keyguard>

Si la tuviésemos, la instalaríamos y ejecutaríamos con adb:

```
~/Desktop $ adb -s 981cd985 install UnlockAndroid.apk
910 KB/s (163224 bytes in 0.174s)
  pkg: /data/local/tmp/UnlockAndroid.apk
Success

~/Desktop $ adb -s 981cd985 shell am start -n com.rohit.unlock/com.rohit.unlock.MainActivity
Starting: Intent { cmp=com.rohit.unlock/.MainActivity }
```

9 Códigos de control de móviles Android

Podemos obtener mucha información a partir de los **códigos de control** que se pueden teclear como un número de teléfono.

Por ejemplo:

- Ver el IMEI del teléfono: *#06 #
- Menú de información: #0# etc.

Más información en: <https://www.malavida.com/es/guias-trucos/lista-completa-con-los-codigos-secretos-de-android-017216>

10 IMEI

Una fuente muy importante de información del teléfono móvil es su número **IMEI (International Mobile Equipment Identity)**.

- Es un código único de 15 dígitos que identifica con precisión cada dispositivo que use una tarjeta SIM en todo el mundo.
- Se creó porque **el número de la tarjeta SIM no puede ser un identificador permanente del dispositivo**.
- La tarjeta SIM está asociada con el usuario y se puede transferir fácilmente de un teléfono a otro.
- Los **primeros 6 dígitos del IMEI son el TAC (Type Approval Code)** y sirve para encontrar el modelo exacto del dispositivo.
- Los primeros 14 dígitos los define la Asociación GSM.

- El último dígito es generado por un algoritmo llamado fórmula de Luhn y es un carácter de control. <https://es.planetcalc.com/2464/>
- Los operadores y fabricantes de móviles usan el IMEI para poder **hacer un seguimiento de los "smartphones" que han sido robados o comprometidos.**

NOTA: No se debe confundir el número IMEI con el número de serie del dispositivo. Los fabricantes usan el número de serie a nivel interno, mientras que el IMEI es un estándar en toda la industria.

Si un número IMEI está bloqueado o en una lista negra, será difícil hacer nada.

- La **lista negra de IMEI** se comparte entre los proveedores de telefonía y no es fácil desbloquear el IMEI.
- Si un teléfono es bloqueado por una red determinada, no funcionará hasta que ésta lo desbloquee.
- La única opción es pedir al proveedor que desbloquee el dispositivo.

10.1 Obtener el IMEI

1. La forma más fácil de verificar el IMEI en cualquier teléfono es usar la **secuencia * # 06 #**.
2. Se puede mirar también a **Settings->About phone->Status->IMEI** o una ruta similar dependiendo del móvil.
3. **Bajo la batería** si esta es extraíble se puede encontrar el IMEI impreso en una pegatina.
4. En la **caja del dispositivo** si disponemos.
5. Con ADB, ejecutando el pedido **"service call iphonesubinfo 4"**



Segundo IMEI (Android 9)

Podríamos obtener el IMEI vía comandos adb.

10.2 Utilidad del IMEI

El número IMEI es útil en las siguientes situaciones:

1. Para **conocer la información técnica oficial**: cantidad de memoria, CPU, placa base, aspecto físico, sistema operativo, etc.
 - Puede ser que no se corresponda al fabricante que pensábamos que era de entrada.
 - Puede ser que el sistema Android instalado no sea la versión que debería tener según las características técnicas.
2. Para obtener **información "secreta"** sobre el producto que a veces permite localizar el teléfono inteligente (a partir de los 6 primeros dígitos).
3. Cuando se quiere **enviar el dispositivo al servicio técnico**, para completar los formularios de garantía.
4. Para **denunciar un teléfono robado o perdido** a la policía o al operador de red.
5. Para **bloquear el teléfono y deshabilitar el dispositivo**, independientemente de si se cambia o se saca la tarjeta SIM.
6. Para **ver si el dispositivo es válido y original (si lo que tenemos es de segunda mano)**.

10.3 Buscar información en Iimei.info

Una de las características más importantes del buscador de **Iimei.info**

(<https://www.imei.info/>) **son los verificadores en línea**, que nos proporcionan:

- la red y el país de donde proviene originalmente su dispositivo
- información de garantía
- el estado de bloqueo de la señal
- informe de listas negras.
- Fecha de compra
- información del operador
- versión del sistema
- especificaciones del dispositivo y más información detallada.

También se pueden obtener **algunos servicios avanzados**, que están disponibles para algunos fabricantes:

- Consultar el Estado del iPhone
- Estado de iCloud
- Estado de lista negra
- Información del vendedor
- comprobar la Red
- Consulta SIMLock
- lista negra de teléfonos
- Verificar Operadores
- Desbloquear SIMLock

También se puede obtener información de: <https://imeicheck.com/es/verificador-imei>

11 Obtener información a partir de la tarjeta SIM

Si tenemos la **tarjeta SIM del teléfono**, podemos obtener información de diferentes maneras:

1. Se puede usar la página <https://informacion-telefonos.com/> y poner el número de móvil para ver proveedor y si está en alguna lista negra.
2. Se puede solicitar el listado de llamadas al proveedor (sólo por vía judicial).
3. Se puede buscar el número en <https://listaspam.com> para ver si está en alguna lista de spam.
4. Se puede utilizar la página <https://tellows.es> para poner el número y saber a quién pertenece (hacer una búsqueda inversa a partir del teléfono).
5. Se puede dar de alta en la agenda de un móvil y consultar en WhatsApp el perfil del número de teléfono.

12 Extracción de datos lógica

Conseguir acceso de **Root no es siempre necesario para obtener una adquisición lógica**. Pero influye mucho en la cantidad y el tipo de datos que se puede extraer.

- Hacer el **Root del dispositivo puede alterar el dispositivo** y se debe hacer solo si tenemos autorización legal para hacerlo.

Hay varias técnicas para realizar una extracción lógica.

12.1 Extracción con ADB pull

Permite copiar ficheros/carpetas del dispositivo.

```
adb pull ruta-remota ruta-local
```

Podríamos extraer todos los datos de las app así:

```
PS C:\Android\platform-tools> .\adb.exe -s emulator-5554 root
PS C:\Android\platform-tools> .\adb.exe -s emulator-5554 pull /data .\ext_logica\
```

Si tenemos algún problema con algún nombre de archivo (por ejemplo, Windows no soporta archivos o carpetas con el carácter :), podemos comprimir la carpeta en el dispositivo y después copiarla:

```
PS C:\Android\platform-tools> .\adb.exe -s emulator-5554 shell tar -czf /sdcard/extraccion_data.tar.gz /data
PS C:\Android\platform-tools> .\adb.exe -s emulator-5554 pull /sdcard/extraccion_data.tar.gz
```

Recuerda borrar la copia comprimida del dispositivo.

Conociendo la estructura de carpetas del sistema Android, podemos adb combinado con diferentes comandos para extraer información:

Encontrar todas las bases de datos sqlite:

```
PS C:\Android\platform-tools> .\adb.exe -s emulator-5554 shell find data -name "*.db" -print |more
```

o desde dentro del dispositivo:

```
emu64xa:/ # find data -name "*.db" -print |more
```

Obtener datos de contactos, telefonía y sms:

```
PS C:\Android\platform-tools> .\adb.exe -s emulator-5554 adb /data/data/com.android.providers.contacts/databases/contacts2.db .
```

```
PS C:\Android\platform-tools> .\adb.exe -s emulator-5554 pull /data/data/com.android.providers.telephony/databases/* .
```

Este último comando obtendrá los ficheros mmssms.db y telephony.db

Obtener datos de navegación y correo electrónico:

```
PS C:\Android\platform-tools> .\adb.exe -s emulator-5554 pull /data/data/com.android.browser/databases/browser2.db
```

Para la los datos de correo electrónico hay que ejecutar el comando siguiente reemplazando CUENTA por el correo electrónico:

```
adb pull  
/data/data/com.google.android.gm/databases/mailstore.<CUENTA>.db
```

Datos de aplicaciones de chat o redes sociales:

- WhatsApp
/data/data/com.whatsapp/databases/msgstore.db
- Facebook
data/data/com.facebook.katana/databases/threads_db2
data/data/com.facebook.orca/databases/threads_db2
- Telegram
/data/data/com.telegram.messenger/databases/cacheX.db
- Hangouts
/data/data/com.google.android.talk/databases/babelX.db
- Tinder
/data/data/com.tinder/databases/tinder.db
- Skype
/data/data/com.skype.raider/files/<NOMBRE_DE_CUENTA>/main.db
- WeChat
/data/data/com.tencent.mm/databases/EnMicroMsg.db
/sdcard/Tencent/MicroMsg
- LINE
/data/data/jp.naver.line.android/databases

12.2 Extracción con ADB backup

Permite hacer un **backup del dispositivo Android**. No hay que ser root, pero el dispositivo debe poder desbloquearse, ya que al empezar lo pide. Hay que ejecutar el comando:

Backup completo

```
adb backup -apk -shared -all -system -f ficherobackupAndroid.ab
PS C:\Android\platform-tools> .\adb.exe -s emulator-5554 backup -apk -shared -all -system -f ficherobackupAndroid.ab
```

Por una sola aplicación, pero solo los datos

```
adb backup -noapk com.your.packagename
```

Si la aplicación es depurable, podemos copiar sus datos sin ser root

```
shell adb
run-as com.your.packagename
cp /data/data/com.tu.nombrepaquete/
```

Los posibles parámetros de adb backup son los siguientes:

- **-f <FICHERO>**: especifica el archivo de copia (por defecto ./backup.ab)
- **-apk | noapk**: habilita o deshabilita la copia de ficheros .apk. Por defecto: -noapk
- **-obb | noobb**: habilita o deshabilita la copia de ficheros adicionales. Por defecto: -noobb
- **-system | nosystem**: incluye las aplicaciones del sistema. Por defecto: -system
- **-shared | noshared**: hace copia de los contenidos de la SD Card y del almacenamiento compartido del dispositivo. Por defecto: -noshared
- **-keyvalue | nokeyvalue**: copia los ficheros .data con formato clave=valor (desde Android 8.0 Oreo)
- **-all**: hace backup de todas las apps instaladas
- **<packages>**: lista de paquetes a copiar (no es necesaria si se usa el flag -all)

Los archivos .ab obtenidos son .tar comprimidos con el algoritmo deflate, opcionalmente cifrados con AES y que incluyen un manifiesto junto con los datos (ficheros APK, BB.DD., etc.). Podemos extraer el fichero tar con la herramienta

<https://github.com/nelenkov/android-backup-extractor>

La descargamos y la ejecutamos con java:

```
PS C:\Android\platform-tools> java -jar abe.jar unpack .\ficherobackupAndroid.ab .\fichero.tar
0% 1% 2% 3% 4% 5% 6% 7% 8% 9% 10% 11% 12% 13% 14% 15% 16% 17% 18% 19% 20% 21% 22% 23% 24% 25% 26% 27% 28% 29% 30% 31% 32% 33% 34% 35% 36% 37% 38% 39% 40% 41% 42% 43% 44% 45% 46% 47% 48% 49% 50% 51% 52% 53% 54% 55% 56% 57% 58% 59% 60% 61% 62% 63% 64% 65% 66% 67% 68% 69% 70% 71% 72% 73% 74% 75% 76% 77% 78% 79% 80% 81% 82% 83% 84% 85% 86% 87% 88% 89% 90% 91% 92% 93% 94% 95% 96% 97% 98% 99% 100%
245760 bytes written to .\fichero.tar.
```

Y ya lo podemos abrir con 7-zip o el propio explorador de Windows.

NOTA: El comando **backup** está obsoleto y puede ser eliminado en versiones posteriores.

12.3 Extracción con Android Studio

También se puede extraer ficheros de los dispositivos con Android Studio >= 3.0 :

- **View > Tool Windows > Device File Explorer**
- Expandir /data/data/[nombre-paquete]

Sólo se puede hacer en paquetes que funcionan en modo debug para dispositivos no-Root.

12.4 Extracción ADB dumphsys (extracción de datos en vivo)

Esta herramienta da información sobre los servicios del dispositivo Android.

1. Lista de servicios: `adb shell dumphsys -l`
2. Información de todos los servicios: `adb shell dumphsys | more`
3. Información de red (usar servicio limpios): `dumphsys netstats`

```
PS C:\Android\platform-tools> .\adb.exe -s DENFMR8DCQ6HX4EY shell
rodin:/ $ dumphsys netstats
Directory:
/data/misc/apexdata/com.android.tethering/netstats
Configs:
netstats_combine_subtype_enabled=false
netstats_store_files_in_apexdata=true
netstats_import_legacy_target_attempts=1
platform legacy stats import attempts count=1
platform legacy stats import successes count=1
platform legacy stats import fallbacks count=0
enable_network_stats_event_logger: true
store_transport_types: true
netstats_fastdatainput_target_attempts=0
FastDataInput successes=0
FastDataInput fallbacks=0
trafficstats.client.cache.config=TrafficStatsRateLimitCacheConfig{isCacheEnabled: true, expiryDurationMs: 1000, maxEntries: 400}
Active interfaces:
iface=ccmni2 ident=[{type=0, ratType=13, subscriberId=214060..., metered=false, defaultNetwork=false, oemManaged=OEM_NONE, subId=1, transports={0}}]
iface=vt_data01 ident=[{type=0, ratType=13, subscriberId=214060..., metered=true, defaultNetwork=true, oemManaged=OEM_NONE, subId=1, transports={1}}]
iface=wlan0 ident=[{type=1, ratType=COMBINED, wifiNetworkKey="ANDROMEDA 5G"wpa2-psk, metered=false, defaultNetwork=true, oemManaged=OEM_NONE, subId=-1, transports={1}}]
Active UID interfaces:
iface=ccmni2 ident=[{type=0, ratType=13, subscriberId=214060..., metered=false, defaultNetwork=false, oemManaged=OEM_NONE, subId=1, transports={0}}]
iface=vt_data01 ident=[{type=0, ratType=13, subscriberId=214060..., metered=true, defaultNetwork=true, oemManaged=OEM_NONE, subId=1, transports={1}}]
iface=wlan0 ident=[{type=1, ratType=COMBINED, wifiNetworkKey="ANDROMEDA 5G"wpa2-psk, metered=false, defaultNetwork=true, oemManaged=OEM_NONE, subId=-1, transports={1}}]
All wifi interfaces:
wlan0
All mobile interfaces:
ccmni2
Top openSession callers:
{uid=1000,package=android}=83
{uid=10155,package=com.microsoft.appmanager}=4
{uid=1000,package=com.miui.securitycenter}=4
```

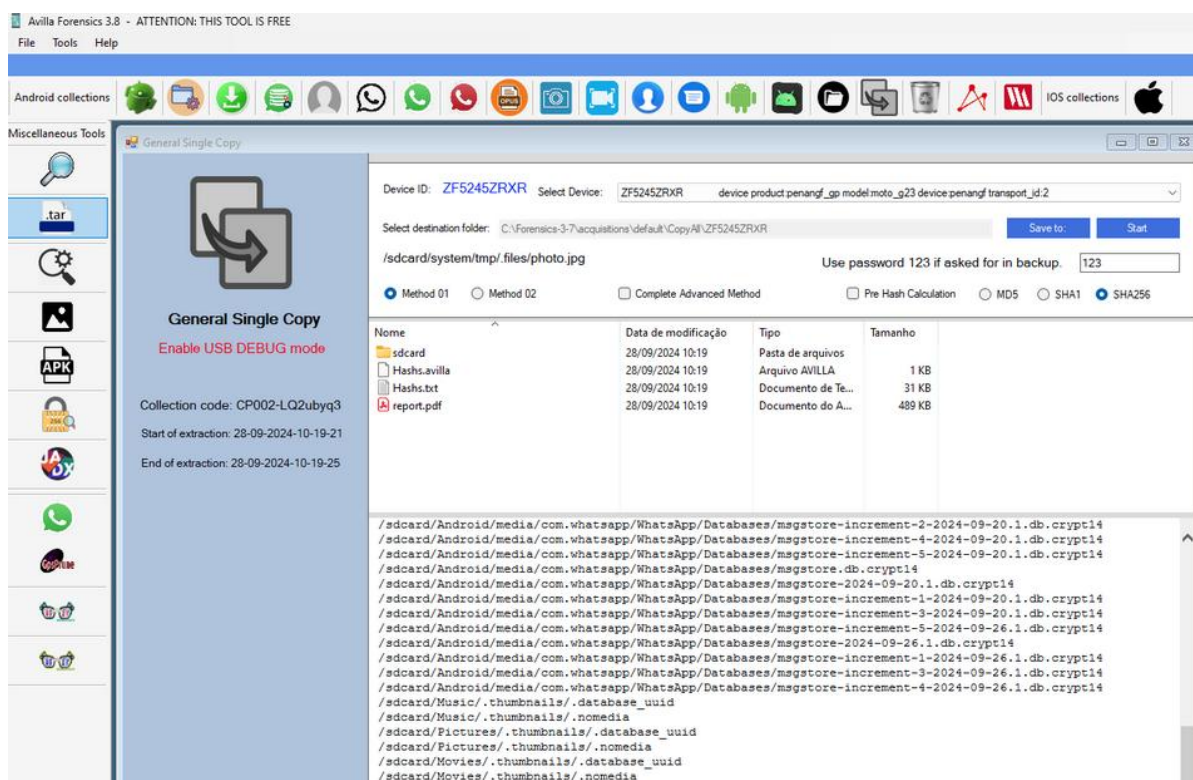
```
adb shell dumphsys netstats detail
```

```
...
```

12.5 Extracción ADB dumpsys (extracción de datos en vivo)

Avilla Forensics es una herramienta forense móvil gratuita creada en febrero de 2021 para ayudar a los investigadores a recoger información y evidencias desde dispositivos móviles. <https://github.com/AvillaDaniel/AvillaForensics>

- Desarrollada por Daniel Avilla, un oficial de policía de S, o Paulo
- permite la extracción de datos lógicas
- Conversión de copias de seguridad en formatos compatibles con software de análisis forense avanzado, como IPED y Cellebrite Physical Analyser.
- Calcula hash automáticamente y guardar de forma encriptada la información.
- Permite la interacción directa con los dispositivos móviles con ADB.
- Permite "APK Downgrade" de más de 15 aplicaciones sin acceso root, explotando vulnerabilidades del sistema Android para acceder a los datos de la aplicación almacenados en la región DATA.
- A partir de la versión 3.8, soporta Android 14 y 15, y explotación de vulnerabilidades en Android 12 y 13 para acceder a información sensible de manera eficiente y segura.
- Permite la adquisición simultánea de datos de múltiples dispositivos
- Dispone de un módulo de Extracción completa, que permite la recogida de datos de cualquier aplicación almacenada en la partición DATA, incluida la información de perfiles secundarios registrados en el dispositivo, sin necesidad de acceso root o desactualizaciones APK.



Avilla Forensics 3.8

12.6 Triage de Android

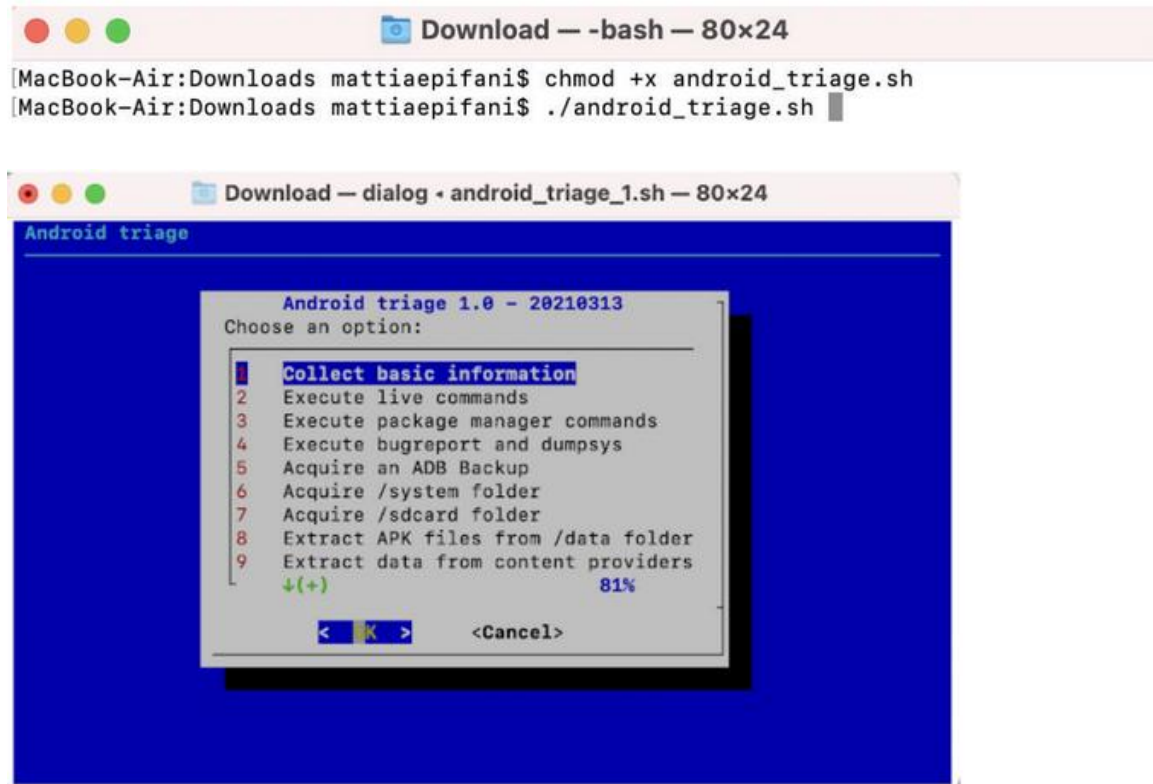
Existen algunas herramientas para realizar triaje automático en Android:

Android Triage

Script en bash preparado para adquirir los artefactos más interesantes

https://github.com/RealityNet/android_triage

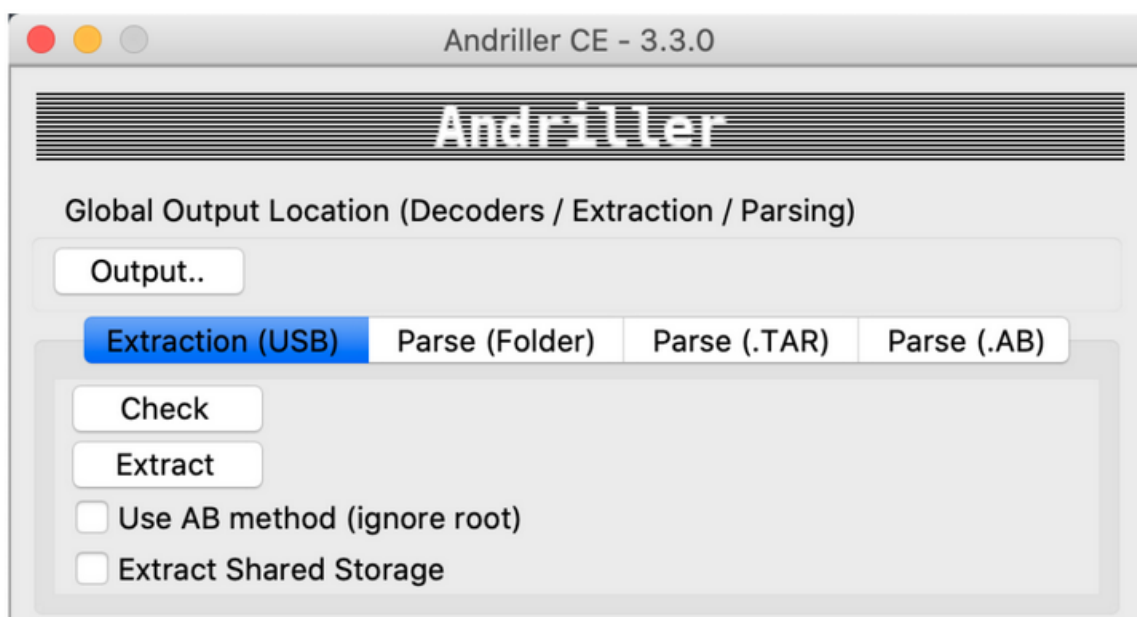
<https://blog.digital-forensics.it/2021/03/triaging-modern-android-devices-aka.html>



Andriller

Herramienta escrita en Python para la extracción de artefactos en dispositivos Android

<https://github.com/den4uk/andriller>



12.7 Usar Cellebrite, AXIOM, Mobiledit

Son herramientas comerciales de pago y por lo tanto no las veremos, excepto una de ellas.

- **Cellebrite UFED** - Acceso a dispositivos móviles de muchos tipos. Accede a pruebas digitales críticas de forma legal a partir de funciones y teléfonos inteligentes, drones, tarjetas SIM, tarjetas SD, dispositivos GPS y más.
 - <https://cellebrite.com/en/ufed/>
- **Magneto Axiom Mobile** - Adquisición Android e iOS y análisis (timeline, Conexiones, explorador de medios, etc.)
 - <https://www.magnefforensics.com/products/magnet-axiom/mobile/>
 - Con versión de prueba.
- **Mobiledit** - Herramienta de software forense integral diseñada para extraer, analizar y generar informes de datos de teléfonos móviles, relojes inteligentes y la nube. Permite a investigadores, peritos y fuerzas de seguridad acceder a información borrada, mensajes, contactos, contraseñas y datos de aplicaciones como WhatsApp, incluso superando bloqueos de seguridad.
 - <https://www.mobiledit.com/mobiledit-forensic>
 - Es la que usaremos en la próxima unidad

13 Preguntas para guiar la investigación

Cuando se examina un dispositivo Android con fines forenses, hay que hacerse las preguntas correctas para guiar la investigación. Hay que priorizar la adquisición de pruebas y preservar la integridad de los datos digitales.

13.1 Preguntas genéricas

Se puede utilizar el método 5Ws y H (del inglés Who, What, When, Where, Why, How):

- ¿Quién usó el dispositivo?
- ¿Qué actividad tuvo lugar?
- ¿Cuándo ocurrió?
- ¿Dónde se utilizó el dispositivo?
- ¿Por qué se realizó la actividad?
- ¿Cómo se ejecutó la actividad (técnicamente)?

13.2 Ejemplos de posibles escenarios

Escenario 1: robo de datos

Supongamos que tenemos un empleado descontento porque no ha sido promocionado debido a un rendimiento bajo. Parece que ha intentado acceder a archivos sensibles, y es sospechoso de haber enviado documentos confidenciales de la empresa a un competidor utilizando aplicaciones de chat.

Teniendo en cuenta este escenario, podemos hacer las siguientes preguntas forenses sobre su dispositivo incautado.

Cuestiones forenses

- ¿Quién tenía acceso a la cuenta de WhatsApp en el dispositivo?
- ¿Qué documentos o archivos se compartieron a través de la app?
- ¿Cuándo se enviaron y recibieron los mensajes?
- ¿Qué contactos estuvieron involucrados en la comunicación?
- ¿Se usaba sólo Whatsapp o se utilizaban otras aplicaciones de comunicación como Telegram o Signal?
- ¿Se tomaron capturas de pantalla o grabaciones de pantalla de información sensible?
- ¿Hay alguna indicación de mensajes borrados o uso de "mensajes que se autodestruyen"?

¿Dónde encontrar las respuestas?

Para encontrar las respuestas a estas preguntas, extraeremos y examinaremos las siguientes evidencias:

- La BD de WhatsApp
- Otras aplicaciones de comunicación
- Evidencia de mensajes eliminados
- Registros (logs) de chat
- Evidencia de presencia o borrado de apps de comunicación
- etc.

Escenario 2: Visitas a sitios web maliciosos o de ciberdelincuencia

Se sospecha que un individuo accede a sitios web oscuros para transacciones ilegales.

Cuestiones forenses

- ¿Qué navegadores o aplicaciones se han utilizado para acceder a la web (Chrome, Firefox, Tor)?
- ¿Qué URL se han accedido y cuál era su contenido?
- ¿Se utilizaban VPN o aplicaciones proxy para enmascarar la ubicación?
- ¿Hay archivos descargados o contenido web almacenados en el dispositivo?
- ¿Cuándo se accedió al contenido sospechoso?
- ¿Hay pruebas de aplicaciones o carteras de criptomonedas?

Dónde encontrar las respuestas

Para encontrar las respuestas a estas preguntas, extraeremos y examinaremos las siguientes evidencias:

- Historial del navegador y actividad web.
- Caché web y descargas.
- Uso de navegadores anónimos o centrados en la privacidad.
- Aplicaciones de comunicación vinculadas al crimen basado en web.

Escenario 3: Infección por malware o compromiso del dispositivo

Se sospecha que un teléfono Android está infectado con software espía.

Cuestiones forenses

- ¿Qué aplicaciones están instaladas? ¿Hay fuentes desconocidas?
- ¿Cuándo se produjo la actividad sospechosa (por ejemplo, sobrecalentamiento, consumo total de la batería)?
- ¿Hay pruebas de acceso remoto o puerta trastienda?
- ¿Qué registros del sistema o cambios de permisos se ha registrado?
- ¿El dispositivo se conectó a IPs extrañas o realizó exfiltración de datos?

¿Dónde encontrar las respuestas?

Para encontrar las respuestas a estas preguntas, extraeremos y examinaremos las siguientes evidencias:

- Lista de aplicaciones instaladas y APKs.
- Permisos y comportamiento de las aplicaciones.
- Servicios desconocidos o en segundo plano.
- Indicios de software malicioso en registros e informes de fallos.
- Comunicación con aplicaciones sospechosas.

En definitiva, cuando investigamos un caso forense que implica un dispositivo Android:

- Debemos entender qué artefactos pueden dejar las huellas de las actividades relacionadas con este escenario.
- Identificamos estos artefactos y los examinamos para encontrar respuestas a las preguntas que podrían ayudar a resolver el caso.
- Es importante formular las preguntas para cualquier investigación forense.